

АННОТАЦИЯ

Выпускная квалификационная работа содержит 83 страницы, 16 рисунков, 10 таблиц, 40 источников, 1 приложение.

Ключевые слова: информационная система, рекомендательная система, персонализация, кбжу, задача о рационе, бинарные отношения предпочтения, гибридное хранилище, матричное разложение, go, vue 3, postgresql, mongodb, python.

Объект исследования – процесс формирования продуктовой корзины пользователя в онлайн-магазинах; предмет – методы и алгоритмы подбора продуктов и блюд с учётом КБЖУ корзины и истории заказов. Цель работы – повышение эффективности формирования рациона путём персонализированного подбора блюд на основе выбранных товарных позиций.

Применены методы математического моделирования (задача о рационе, теория бинарных отношений предпочтения, матричное разложение TruncatedSVD), методы программной инженерии (многослойная клиент-серверная архитектура, гибридное хранилище данных) и методы оценки качества рекомендаций (Precision@K, Recall@K, авторская метрика NAS).

В результате спроектирована и реализована информационная система с гибридным хранилищем PostgreSQL и MongoDB, серверной частью на Go (Gin, GORM), клиентской частью на Vue 3 с TypeScript и аналитическим модулем на Python (scikit-learn) с двухслойным гибридным модулем рекомендаций.

Научная новизна – двухслойный подход к подбору рациона с модифицированной функцией полезности и введённая авторская метрика NAS. Практическая значимость – возможность встраивания системы в существующие онлайн-сервисы продуктовой розницы как модуля контроля рациона.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	9
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ	12
1.1 Исследование проблемы контроля рациона питания	12
1.2 Обзор существующих информационных систем	13
1.2.1 Рекомендательные механизмы в сервисах электронной торговли продуктами	13
1.2.2 Приложения для мониторинга нутриентов	13
1.2.3 Сервисы доставки готовых рационов	14
1.2.4 Сравнительный анализ	14
1.3 Математическая постановка задачи о рационе	15
1.3.1 Классическая модель задачи о диете	15
1.3.2 Целевые нормы базового профиля пользователя	16
1.3.3 Обзор методов решения и их применимости	16
1.4 Формализация предпочтений пользователя	18
1.4.1 Бинарные отношения предпочтения	18
1.4.2 Иерархия предпочтений в контексте здорового питания	19
1.4.3 Источники сигналов о предпочтениях	19
1.4.4 Функция полезности	20
2 Проектирование информационной системы	23
2.1 Моделирование бизнес-процессов	23
2.1.1 Текущий процесс (As-Is)	23
2.1.2 Целевой процесс (To-Be)	24
2.1.3 Концепция гибридного хранения	25

2.2 Разработка технических требований.....	25
2.2.1 Функциональные требования.....	26
2.2.2 Нефункциональные требования	27
2.2.3 Ролевая модель	28
2.3 Проектирование архитектуры.....	28
2.3.1 Общая схема архитектуры	28
2.3.2 Структура данных	30
2.3.3 Логика взаимодействия компонентов.....	32
2.3.4 Многослойная структура серверной части.....	33
2.4 Обоснование выбора технологического стека	35
2.4.1 Серверная часть: язык Go.....	35
2.4.2 Клиентская часть: Vue 3	36
2.4.3 Хранилища данных	36
2.4.4 Модуль машинного обучения: Python	37
3 ТЕХНИЧЕСКАЯ РЕАЛИЗАЦИЯ СИСТЕМЫ	41
3.1 Разработка структуры базы данных	41
3.1.1 Гибридная модель хранения	41
3.1.2 Логическое проектирование.....	41
3.1.3 Детальная спецификация ключевых таблиц	41
3.1.4 Документоориентированное хранилище изображений	43
3.1.5 Стратегия индексации	43
3.2 Реализация серверной части	44
3.2.1 Точка входа и инициализация.....	44
3.2.2 Регистрация маршрутов и middleware	44
3.2.3 Обобщённый CRUD-хендлер.....	45

3.2.4 Аутентификация и авторизация.....	45
3.2.5 Работа с базами данных	46
3.2.6 Управление каталогом, заказами и блюдами	46
3.3 Реализация клиентской части	47
3.3.1 Структура одностраничного приложения	47
3.3.2 Слой взаимодействия с серверным API.....	47
3.3.3 Представления (views).....	47
3.3.4 Управление состоянием и реактивная корзина.....	48
3.3.5 Загрузка изображений товаров	49
3.4 Реализация рекомендательного модуля.....	50
3.4.1 Эвристический слой: подбор блюд по корзине	50
3.4.2 Аналитический слой: персональные рекомендации	52
3.4.3 Интеграция Go с Python.....	53
3.4.4 Связь с целями работы.....	54
3.5 Интерфейс системы и демонстрация работы	55
3.5.1 Авторизация и регистрация	55
3.5.2 Каталог продуктов с КБЖУ	56
3.5.3 Корзина и автоматический расчёт КБЖУ	57
3.5.4 Рекомендации и подбор блюд.....	58
4 АНАЛИЗ ЭФФЕКТИВНОСТИ И ТЕСТИРОВАНИЕ.....	61
4.1 Тестирование системы.....	61
4.1.1 Модульное тестирование.....	61
4.1.2 Интеграционное тестирование	62
4.1.3 Нагрузочное тестирование	62
4.1.4 Тестирование клиентской части	63

4.1.5 Оценка качества рекомендаций	64
4.2 Оценка качества рекомендаций	65
4.2.1 Методология	65
4.2.2 Сравнение моделей	66
4.2.3 Аналитическое обоснование гибридного скоринга.....	67
4.2.4 Интерпретация результатов	67
4.3 Оценка эффективности реализации	69
4.3.1 Техническая эффективность	69
4.3.2 Социальный эффект	69
4.3.3 Бизнес-показатели	69
4.3.4 Масштабируемость	70
ЗАКЛЮЧЕНИЕ	73
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	75
ПРИЛОЖЕНИЕ	79

ВВЕДЕНИЕ

Тема работы связана с задачами государства в области общественного здравоохранения. Национальный проект «Продолжительная и активная жизнь» [1] предполагает, что к 2030 году средняя продолжительность жизни в России вырастет до 78 лет. Один из путей к этому – улучшение культуры питания. Однако несбалансированный по калориям, белкам, жирам и углеводам рацион остаётся причиной многих хронических заболеваний [2]. Существующие сервисы помогают слабо: онлайн-магазины и сервисы доставки продуктов не считают КБЖУ корзины, а фитнес-приложения требуют ручного ввода каждой позиции уже после покупки.

Объект исследования – процесс формирования продуктовой корзины пользователя в онлайн-магазинах продуктов. Предмет исследования – методы и алгоритмы подбора продуктов и блюд с учётом КБЖУ корзины и истории заказов пользователя.

Тема работы (утверждена заданием на выполнение выпускной квалификационной работы магистра ЭФМО-02-24 от 09.02.2026): «Разработка информационной системы для персонализированного подбора товарных позиций на основе предпочтений пользователя».

Цель работы – повышение эффективности формирования рациона питания граждан путём персонализированного подбора блюд на основе выбранных товарных позиций.

Для достижения поставленной цели в работе решаются следующие задачи (в формулировке, утверждённой заданием на ВКР):

- исследование предметной области и выявление объекта и предмета исследования;
- рассмотреть задачу о рационе и методы её решения;
- рассмотреть бинарные отношения предпочтения в заданной предметной области;

- разработать технические требования к информационной системе и её архитектуру;
- выполнить адаптацию задачи о рационе в заданной предметной области;
- определить оценку эффективности реализации информационной системы.

Для краткости далее используется сокращение КБЖУ – калорийность, белки, жиры и углеводы. На практике все задачи реализованы в виде информационной системы. У неё гибридное хранилище данных (PostgreSQL и MongoDB), серверная часть на языке Go, клиентская часть в виде одностраничного приложения на Vue 3 и двухслойный модуль рекомендаций. Первый слой работает на Go и подбирает блюда по простым правилам, второй – на Python и использует матричное разложение TruncatedSVD для персонализации.

Научная новизна работы – в предложенном двухслойном подходе к подбору рациона. Первый слой написан на Go и считает оценку блюда по составу корзины: какая доля ингредиентов уже есть, можно ли собрать блюдо целиком, и насколько КБЖУ блюда близко к норме на один приём пищи. Второй слой написан на Python и подбирает товары лично под пользователя. Он объединяет четыре оценки: похожесть на товары, которые покупают похожие пользователи (через матричное разложение TruncatedSVD), близость КБЖУ товара к целевому профилю, связь товара с подходящими блюдами и штраф за то, что товар уже недавно покупался. Такой подход не сводится к чистой задаче о диете в постановке линейного программирования. За счёт этого он учитывает и нормы КБЖУ, и личные предпочтения пользователя.

Практическая значимость работы – в том, что разработанная система может встраиваться в существующие онлайн-сервисы по продаже продуктов как модуль контроля рациона. Тем самым она закрывает разрыв между процессом покупки и контролем КБЖУ корзины.

Методы исследования: анализ предметной области, математическое моделирование (классическая задача о диете, теория бинарных отношений предпочтения, матричное разложение), методы программной инженерии (многослойная архитектура, гибридное хранилище, межпроцессная интеграция) и методы оценки качества рекомендаций (Precision@K, Recall@K).

Работа состоит из введения, четырёх глав, заключения и списка источников. В первой главе разбирается предметная область, обзор аналогов, математическая постановка задачи о рационе и описание предпочтений пользователя. Во второй главе сформулированы технические требования, спроектирована архитектура и обоснован выбор технологий. В третьей главе описана техническая реализация: базы данных, серверная часть на Go, клиентская часть на Vue 3 и модуль рекомендаций. В четвёртой главе приведены результаты тестирования и оценка эффективности.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ

1.1 Исследование проблемы контроля рациона питания

Актуальность работы связана с задачами государства в области здравоохранения. По нацпроекту «Продолжительная и активная жизнь» к 2030 году средняя продолжительность жизни в России должна вырасти до 78 лет. Чтобы этого достичь, нужно менять и образ жизни граждан, в том числе культуру питания.

Сейчас сложилась интересная ситуация. Купить продукты через интернет стало очень легко: онлайн-магазины и службы доставки доступны почти всем. Но при этом у пользователя нет удобного способа понимать, что именно по нутриентам лежит в его корзине. В итоге рацион получается несбалансированным по калориям, белкам, жирам и углеводам, и это становится фактором риска для здоровья. Поэтому нужны цифровые помощники, которые подсказывают пользователю, что выбрать прямо в момент покупки.

В рамках работы разрабатывается информационная система такого типа. От типовых рекомендательных сервисов «похожих товаров» она отличается тем, что подбирает не просто похожие продукты, а целые блюда – с учётом и состава корзины, и истории заказов. Основные функции:

- анализ истории заказов и избранных товаров – чтобы понять, что пользователь предпочитает;
- анализ корзины в реальном времени – при каждом изменении система подбирает блюда, которые можно приготовить из уже выбранных товаров, и подсказывает, чего не хватает;

- подбор по нормам КБЖУ – система учитывает простую базовую норму: 2500 ккал в сутки для мужчин и 2000 ккал для женщин, делённые на три приёма пищи.

Таким образом, разрабатываемая система не просто автоматизирует покупку, а помогает пользователю заранее формировать сбалансированный рацион. Это соответствует общим задачам цифровизации в сфере здравоохранения.

1.2 Обзор существующих информационных систем

В рамках исследования был проведён обзор информационных систем, прямо или косвенно решающих задачу формирования рациона питания. На российском рынке выделяются три основных сегмента: сервисы онлайн-ритейла, мобильные приложения для трекинга нутриентов и сервисы доставки готовых рационов.

1.2.1 Рекомендательные механизмы в сервисах электронной торговли продуктами

Крупные российские сервисы доставки продуктов («ВкусВилл», «СберМаркет», «Самокат», «Яндекс Лавка») активно используют рекомендательные алгоритмы. Чаще всего это коллаборативная фильтрация и правила вида «с этим товаром часто покупают». Главная цель таких систем – увеличить средний чек и количество позиций в корзине. КБЖУ продуктов в этих алгоритмах не учитывается, поэтому для задач здорового питания они не подходят.

1.2.2 Приложения для мониторинга нутриентов

Сервисы FatSecret, Lifesum и YAZIO учитывают калорийность и баланс белков, жиров и углеводов уже съеденных блюд. Их главное ограничение –

пассивная роль: пользователь вносит данные уже после еды, и каждую позицию приходится искать вручную и вводить вес порции. Это утомляет, и большая часть пользователей быстро бросает приложение. Связи между планированием покупки и итоговым отчётом о питании в таких сервисах нет.

1.2.3 Сервисы доставки готовых рационов

Компании Grow Food и Level Kitchen предлагают готовые рационы: пользователь получает уже приготовленные блюда с заранее рассчитанным КБЖУ. У таких сервисов два минуса. Первый – мало гибкости: пользователь не может оперативно поменять меню. Второй – высокая стоимость, которая делает их недоступными большинству.

1.2.4 Сравнительный анализ

Сравнительная характеристика рассмотренных сегментов приведена в Таблице 1.1.

Таблица 1.1 — Сравнительная характеристика существующих типов информационных систем по подбору питания

Критерий	Электронные продуктовые сервисы	Фитнес-трекеры	Сервисы готовых рационов
Целевая ориентация	Коммерческая (продажи)	Информационная (учёт)	Продуктовая (готовое блюдо)
Автоматизация подбора	Есть (без учёта КБЖУ)	Отсутствует (ручной ввод)	Полная (без гибкости)
Учёт состава корзины	Отсутствует	Отсутствует	Не применимо
Доступность	Высокая	Средняя	Низкая (высокая цена)

Из обзора видно, что между обычным онлайн-магазином продуктов и диетологией есть технологический разрыв. Коммерческие сервисы игнорируют КБЖУ, а сервисы учёта питания требуют ручного ввода. Это значит, что нужна система, которая объединяет функции каталога продуктов

и помощника по подбору рациона. Такая система должна анализировать состав корзины прямо в момент покупки. Схематически выявленные информационные разрывы представлены на Рисунке 1.1.



Рисунок 1.1 — Схема информационных разрывов в процессе формирования рациона

Из этой схемы и сравнения видно: ни один из существующих сервисов не объединяет каталог товаров, автоматический расчёт КБЖУ и подбор блюд по корзине. Поэтому и нужна отдельная информационная система с такими функциями – её разработка и есть цель настоящей работы.

1.3 Математическая постановка задачи о рационе

Задача о рационе (или задача о диете) – одна из классических задач оптимизации. Впервые её поставил Дж. Стиглер в 1945 году [3]. В данной работе она используется как теоретическая основа, позволяющая перейти от случайного выбора продуктов к более обоснованному.

1.3.1 Классическая модель задачи о диете

В классической постановке задача о рационе формулируется как задача линейного программирования [4]. Её цель – найти такой набор продуктов, чтобы общая стоимость была минимальной, а норма питательных веществ выполнялась.

Пусть n – количество доступных продуктов, m – количество нутриентов. Тогда целевая функция имеет вид:

$$\min \sum_{j=1}^n c_j \cdot x_j \quad (1.1)$$

при следующих ограничениях:

$$\sum_{j=1}^n a_{ij} \cdot x_j \geq b_i, \quad i = 1..m; \quad x_j \geq 0 \quad (1.2)$$

где x_j – количество единиц продукта j ; c_j – стоимость единицы продукта j ; a_{ij} – содержание нутриента i в продукте j ; b_i – суточная норма нутриента i .

1.3.2 Целевые нормы базового профиля пользователя

В системе используется упрощённая модель «среднего пользователя» – она зависит только от пола. По рекомендациям Роспотребнадзора [5] и ВОЗ [6], для взрослого мужчины с умеренной активностью суточная норма – около 2500 ккал, для женщины – около 2000 ккал. В коде сервера это записано константами `dailyCaloriesMale = 2500` и `dailyCaloriesFemale = 2000` в файле `internal/services/meal_from_order.go`. Суточная норма делится на три приёма пищи (`mealsPerDay = 3`), и получается целевая калорийность одного приёма: 833,3 ккал для мужчин и 666,7 ккал для женщин.

Целевые доли макронутриентов в энергии одного приёма пищи в коде заданы так: белки – 25%, жиры – 30%, углеводы – 45%. Это соответствует рекомендациям ВОЗ и используется в системе как ориентир при оценке баланса корзины (см. пункт 3.4.1).

1.3.3 Обзор методов решения и их применимости

Для подбора продуктов в информационных системах применяют разные группы методов:

- точные методы, например симплекс-метод: хорошо работают для небольших каталогов, но при сотнях и тысячах товаров становятся слишком медленными и плохо учитывают нелинейные условия – например, сочетаемость продуктов или их вхождение в блюда;

- приближённые методы – эвристики и метаэвристики (генетические алгоритмы, имитация отжига): они быстро находят неидеальное, но достаточно хорошее решение и могут учитывать нелинейные критерии, в том числе предпочтения пользователя;
- методы целочисленного программирования: подходят онлайн-магазинам, где товары не делятся на дробные части.

Получается, что классическая модель линейного программирования нужна как теоретическая основа, но в чистом виде её недостаточно. Она учитывает только цену и нормы нутриентов, а в онлайн-магазине продуктов важны ещё и предпочтения пользователя, история заказов, и то, как товары сочетаются друг с другом в составе блюд. Поэтому в практической части (см. главу 3) рекомендательный модуль построен по другой схеме: оценка состава корзины и блюд по простым правилам в сервисе на Go плюс матричное разложение и контент-ориентированный подбор в модуле на Python. Сводное сравнение перечисленных методов приведено в Таблице 1.2.

Таблица 1.2 — Сравнительная характеристика методов решения задачи о рационе

Метод	Скорость	Точность	Учёт нелинейных ограничений	Применимость
Симплекс-метод (LP)	Средняя	Высокая (для малых каталогов)	Нет	Каталоги до сотен позиций
Метод ветвей и границ (целочисленное программирование)	Низкая	Высокая	Частично	Дискретные каталоги онлайн-магазинов малого объёма
Генетические алгоритмы	Средняя	Субоптимальная	Да	Большие каталоги, нелинейные критерии
Эвристический скоринг (применён в Go-сервисе, см. п. 3.4.1)	Высокая	Приемлемая	Да	Реальное время, отклик ≤ 200 мс

Продолжение Таблицы 1.2

Матричное разложение TruncatedSVD (применено в Python-модуле, см. п. 3.4.2)	Высокая	Зависит от объёма истории	Да (через скрытые факторы)	Персонализация по истории заказов
---	---------	---------------------------	----------------------------	-----------------------------------

Из Таблицы 1.2 видно, что для решаемой задачи лучше всего подходит комбинированный подход: классическая модель линейного программирования служит теоретической базой для целевых нутриентных норм, а практический подбор товаров и блюд делают рекомендательные алгоритмы. Они умеют учитывать нелинейные критерии – предпочтения пользователя, сочетаемость ингредиентов в блюде и сезонную доступность товаров.

1.4 Формализация предпочтений пользователя

Чтобы перейти от классической модели к персонализированной информационной системе, нужно формально описать пользовательские предпочтения. В работе для этого применяется аппарат бинарных отношений на множестве товаров и блюд.

1.4.1 Бинарные отношения предпочтения

Пусть A – конечное множество объектов системы (товаров и блюд). На паре элементов из A определяются три отношения: строгое предпочтение P (запись xPy означает « x лучше y »), безразличие I (xIy – « x и y одинаково подходят») и нестрогое предпочтение R (xRy – « x не хуже y »). Для нормальной работы рекомендательного модуля считаем, что отношение полное (любые два элемента можно сравнить) и транзитивное (если xPy и yPz , то xPz) [7].

На практике для пищевых предпочтений транзитивность часто нарушается. Например, продукт A может быть приятнее B , B – приятнее C , но C при этом приятнее A в зависимости от настроения, времени суток и других

обстоятельств. Поэтому в системе применяются вероятностные модели выявления предпочтений (см. пункт 1.4.3).

1.4.2 Иерархия предпочтений в контексте здорового питания

В рамках задачи поддержки здорового рациона предпочтения упорядочены иерархически:

- жёсткие ограничения (аллергии, медицинские противопоказания) – исключают позиции из рассмотрения, имеют приоритет над всеми остальными уровнями;
- диетические предпочтения (вегетарианство, низкоуглеводная диета и тому подобное) – определяют допустимое подмножество товаров;
- вкусовые (гедонистические) предпочтения – выявляются по истории заказов и сохранённым избранным позициям.

В реализованной системе нижний уровень иерархии – вкусовые предпочтения – отражён в таблицах `favourite_products` и `favourite_dishes`. Они хранят простое отношение «отмечено как избранное». Жёсткие ограничения и диетические предпочтения в текущей версии в базе не сохраняются – это направление дальнейшего развития системы.

1.4.3 Источники сигналов о предпочтениях

Система использует два типа сигналов о предпочтениях пользователя:

- явные предпочтения: отметка товара или блюда как избранного через эндпоинты `/v1/favourites/*` серверной части;
- неявные предпочтения: анализ истории заказов пользователя (таблицы `orders` и `order_items`) и состава его текущей корзины, передаваемой в эндпоинт `/v1/users/:id/meals/from-cart`.

1.4.4 Функция полезности

Бинарные отношения предпочтения позволяют изменить целевую функцию задачи о рационе. Похожий подход с заменой минимизации стоимости на максимизацию функции полезности подробно описан в работе [8]. Вместо минимизации стоимости система переходит к максимизации функции полезности:

$$U(x) = \sum w_j \cdot x_j \cdot \mu_j \quad (1.3)$$

где w_j – весовой коэффициент, который показывает ранг товара в системе предпочтений пользователя (он рассчитывается по данным `favourite_products` и `order_items`). Множитель μ_j – это индекс того, насколько КБЖУ товара соответствует текущему дефициту пользователя. В реальной реализации (см. пункт 3.4) роль w_j играют коллаборативная и контент-ориентированная компоненты Python-модуля рекомендаций, а μ_j – компонента нутриентной оценки блюд: она оценивает, насколько товар повышает шанс собрать сбалансированное блюдо из текущей корзины. Пример графа отношений предпочтения для одной из категорий каталога приведён на Рисунке 1.2.



Рисунок 1.2 — Граф отношений предпочтения для категории «Молочные продукты»

Этот граф показывает, как отношения предпочтения превращаются в логику работы рекомендательного модуля. Жёсткие ограничения (полное

исключение позиций) делаются на этапе фильтрации кандидатов, а вкусовые и диетические предпочтения – через весовые коэффициенты в функции полезности.

По итогам первой главы можно сделать следующие выводы:

- проанализирована проблема контроля рациона питания и обоснована её актуальность в контексте национального проекта «Продолжительная и активная жизнь»;
- проведён сравнительный обзор существующих информационных систем – сервисов электронной торговли продуктами, фитнес-трекеров и сервисов готовых рационов – и выявлен информационный разрыв между процессом покупки и контролем нутриентного состава корзины;
- сформулирована классическая математическая постановка задачи о рационе в виде задачи линейного программирования и обоснованы целевые нормы базового профиля пользователя (2500 ккал в сутки для мужчин и 2000 ккал в сутки для женщин, с разделением на три приёма пищи и долями белков, жиров и углеводов 25, 30 и 45 процентов);
- проведён сравнительный анализ методов решения задачи (симплекс-метод, метаэвристические алгоритмы, целочисленное программирование) и обоснован переход к гибридной схеме, сочетающей эвристический скоринг и аналитические методы машинного обучения;
- формализованы пользовательские предпочтения через теорию бинарных отношений (строгое предпочтение P , безразличие I , нестрогое предпочтение R), построена иерархия предпочтений (жёсткие ограничения, диетические предпочтения, вкусовые предпочтения) и предложена модифицированная функция полезности $U(x)$, интегрирующая нутриентный и предпочтенческий критерии.

Полученная математическая модель и описанная структура бинарных отношений предпочтения служат основой для следующего этапа –

проектирования архитектуры информационной системы, которому посвящена глава 2.

Главный вывод первой главы: классическая задача о диете в виде задачи линейного программирования сама по себе недостаточна для современной системы поддержки рациона. Её решение оптимально по цене и нормам нутриентов, но не учитывает предпочтения пользователя, сочетаемость товаров в составе блюд и сезонную доступность позиций каталога. Чтобы это исправить, в работе предложен двухкомпонентный подход. Классическая модель используется только как источник целевых нутриентных ограничений, а ранжирование выполняется по функции полезности $U(x)$, которая объединяет нутриентный индекс μ_j и весовой коэффициент w_j предпочтений пользователя.

Такой подход согласуется с современной практикой проектирования рекомендательных систем для онлайн-магазинов: статистические методы и простые правила дополняют, а не заменяют классические оптимизационные модели [9]. Новизна работы – в том, что нутриентный индекс μ_j встроен прямо в функцию ранжирования товаров. Это отличает предлагаемое решение от типовых сервисов, где анализ КБЖУ реализован отдельно от каталога и работает только постфактум.

Стоит дополнительно отметить практическую важность иерархии предпочтений. На практике пользователь редко может сам в цифрах задать своё пищевое поведение, поэтому сочетание явных сигналов (отметка избранного) и неявных (анализ истории заказов и текущей корзины) даёт более устойчивую модель, чем каждый источник по отдельности.

Теоретические результаты первой главы – формальная постановка задачи о рационе, иерархия предпочтений и модифицированная функция полезности – задают понятийный аппарат для остальных глав. В главе 2 на их основе формулируются требования к системе и проектируется её архитектура, в главе 3 реализуются конкретные алгоритмы скоринга и рекомендаций, а в главе 4 оцениваются результаты тестирования и качество выдачи.

2 ПРОЕКТИРОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ

Этап проектирования связывает теоретическое обоснование с программной реализацией. На этом этапе формализуются бизнес-процессы, технические требования, архитектура приложения и обоснование выбора технологий [10].

2.1 Моделирование бизнес-процессов

Для обоснования необходимости интеллектуальной поддержки выбора продуктов и визуализации изменений в логике работы сервиса были смоделированы текущий и целевой бизнес-процессы в нотации BPMN [11].

2.1.1 Текущий процесс (As-Is)

В типовом сервисе электронной торговли продуктами информационная система выполняет пассивную роль: загружает каталог по запросу пользователя и принимает оформленный заказ. Ответственность за сбалансированность корзины полностью лежит на пользователе, никакой проверки соответствия её состава нормам КБЖУ не производится. Соответствующая диаграмма бизнес-процесса “As-Is” приведена на Рисунке 2.1.

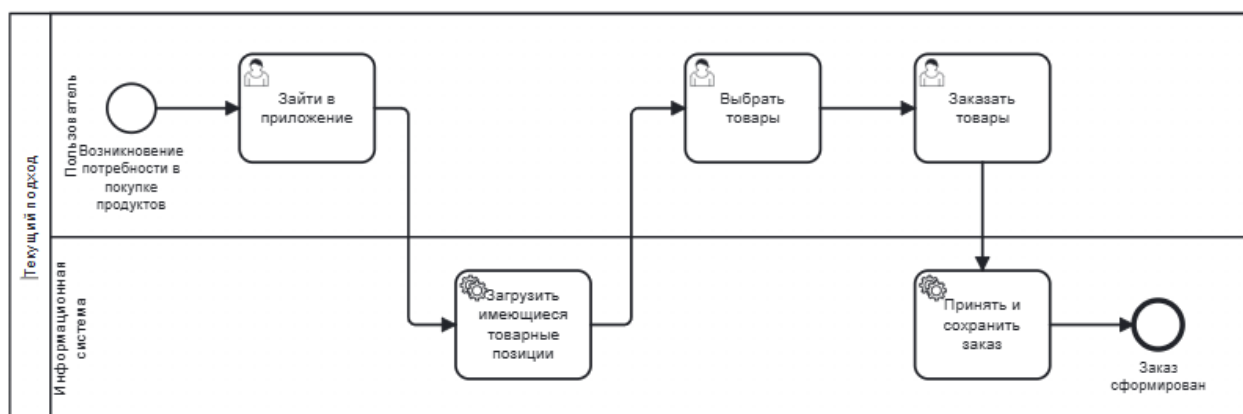


Рисунок 2.1 — Диаграмма бизнес-процесса “As-Is” в нотации BPMN

Представленная диаграмма наглядно показывает, что в существующем процессе все нутриентные расчёты выполняются пользователем самостоятельно, а информационная система играет лишь роль каталога и платёжного шлюза. Это и приводит к информационному разрыву, который устраняется в целевом процессе “To-Be”.

2.1.2 Целевой процесс (To-Be)

Предлагаемое решение интегрирует в процесс покупки контур обратной связи и модуль генерации рекомендаций. После каждого изменения состава корзины серверный модуль (эндпоинт `/v1/users/:id/meals/from-cart`) выполняет следующие шаги: подгружает справочник блюд с предзагрузкой состава; сопоставляет содержимое корзины с требуемыми ингредиентами каждого блюда; вычисляет долю совпавших ингредиентов и нутриентный скор блюда; формирует список недостающих ингредиентов с предложениями товаров из каталога. Параллельно модуль персональных рекомендаций (эндпоинты `/v1/users/:id/recommendations/*`) учитывает историю заказов и избранных позиций. Диаграмма целевого процесса “To-Be” приведена на Рисунке 2.2.

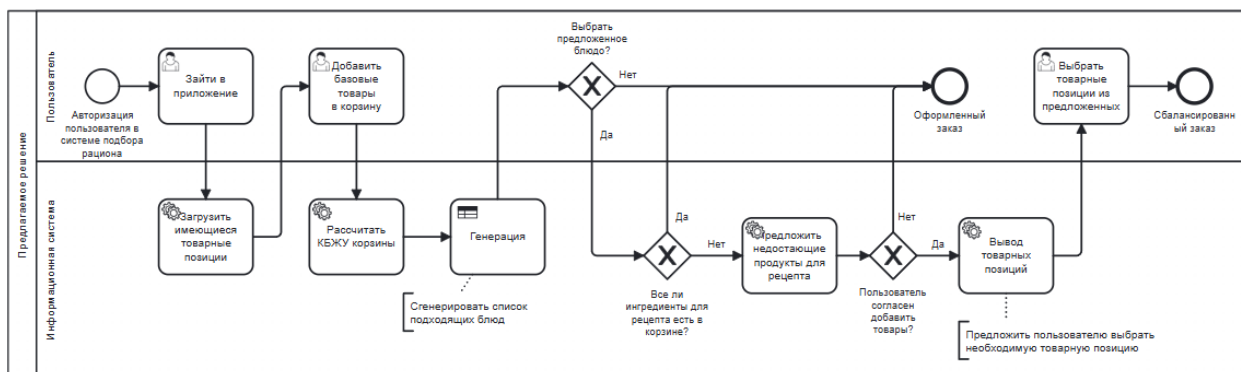


Рисунок 2.2 — Диаграмма бизнес-процесса “То-Ве” в нотации BPMN

В целевом процессе ключевые трудоёмкие действия – расчёт КБЖУ корзины, проверка собираемости блюд и подбор недостающих ингредиентов – выносятся на сторону серверного модуля, что снимает с пользователя необходимость держать в памяти нормы и ингредиентные карты. Это превращает каталог из пассивного списка товаров в активного помощника по формированию рациона.

2.1.3 Концепция гибридного хранения

Для совмещения требований по целостности расчётов нутриентного баланса и эффективному хранению медиа-контента спроектирована гибридная архитектура хранения. PostgreSQL используется как транзакционный слой для всех структурированных данных: профилей пользователей, каталога товаров с подробными нутриентными характеристиками, заказов, блюд и их составов, избранных позиций. MongoDB используется для хранения бинарных изображений товаров. Связь между хранилищами реализована не через идентификатор изображения в PostgreSQL, а через идентификатор товара `product_id`, который фигурирует как поле в каждом документе коллекции изображений MongoDB.

2.2 Разработка технических требований

Требования к системе разделены на функциональные (что система делает) и нефункциональные (как она это делает). Перечень функциональных

требований приведён в Таблице 2.1; они отражают именно тот функционал, который реализован в коде.

2.2.1 Функциональные требования

Таблица 2.1 — Функциональные требования к информационной системе

Код	Категория	Описание требования
FR-01	Регистрация и аутентификация	Система должна обеспечивать регистрацию пользователя по имени, паролю и полу (male/female) и аутентификацию с выдачей JWT-токена (эндпоинты POST /register, POST /login).
FR-02	Управление каталогом	Система должна предоставлять операции создания, чтения, изменения и удаления над товарами, категориями, производителями и блюдами; операции изменения каталога доступны только пользователю с ролью admin.
FR-03	Хранение изображений	Система должна поддерживать загрузку и выдачу бинарных изображений товаров с хранением в MongoDB (эндпоинты POST и GET /v1/products/:id/image).
FR-04	Формирование заказа	Система должна позволять пользователю формировать корзину на клиенте (localStorage) и оформлять её в заказ через эндпоинты /v1/orders и /v1/order-items.
FR-05	Подбор блюд по корзине	Система должна по составу корзины формировать ранжированный список блюд с указанием доли совпавших ингредиентов, признака собираемости, расчётного КБЖУ блюда и перечня недостающих ингредиентов с подсказками-товарами (эндпоинт POST /v1/users/:id/meals/from-cart).
FR-06	Подбор блюд по заказу	Система должна формировать список блюд, которые можно приготовить из ранее оформленного заказа (эндпоинт GET /v1/users/:id/meals/from-order/:orderId).
FR-07	Персональные рекомендации	Система должна выдавать персональные рекомендации товаров и блюд по эндпоинтам /v1/users/:id/recommendations/products, /dishes, /final, используя гибридный алгоритм на основе TruncatedSVD и контент-ориентированных признаков.
FR-08	Избранное	Система должна позволять добавлять и удалять товары из списка избранного пользователя (эндпоинты /v1/favourites/products).

Продолжение Таблицы 2.1

FR-09	Администрирование	Система должна предоставлять администратору доступ к списку пользователей и расширенной информации о них (эндпоинты /v1/users/full и /v1/users/:id/full).
-------	-------------------	---

Представленный перечень требований определяет полный сценарий взаимодействия пользователя с системой и служит основой для проектирования архитектуры и интерфейса.

2.2.2 Нефункциональные требования

- производительность: время отклика серверной части на типичные операции каталога и корзины не должно превышать 200 мс, время отклика рекомендательных эндпоинтов с вызовом Python-скрипта — до 500 мс;
- масштабируемость: stateless-архитектура Go-сервиса позволяет горизонтально масштабировать серверную часть; работа с MongoDB и PostgreSQL ведётся через стандартные драйверы, поддерживающие пул соединений;
- надёжность: ACID-гарантии PostgreSQL обеспечивают согласованность данных о пользователях, заказах и составе блюд; авто-миграция моделей GORM при старте предотвращает рассинхрон схемы;
- безопасность: пароли пользователей хранятся в виде bcrypt-хешей; аутентификация выполняется через JWT-токен, передаваемый в заголовке Authorization; операции изменения каталога защищены middleware adminOnly;
- адаптивность интерфейса: клиентская часть на Vue 3 реализована как одностраничное приложение и корректно отображается на устройствах различных классов.

2.2.3 Ролевая модель

В системе предусмотрены две роли пользователей: “user” (по умолчанию для зарегистрированных через эндпоинт /register) и “admin” (создаётся при первом запуске сервиса из переменных окружения ADMIN_NAME и ADMIN_PASSWORD). Роль admin отличается доступом к операциям создания, изменения и удаления над товарами, категориями, производителями и блюдами, а также к расширенной информации о пользователях. Контроль доступа реализован middleware authMiddleware (проверка JWT) и adminOnly (проверка роли в claims) в файле internal/api/v1/apies.go.

2.3 Проектирование архитектуры

Архитектура системы построена по клиент-серверной модели с разделением на три уровня: клиентская часть (одностраничное приложение на Vue 3), серверная часть (Go-приложение с многослойной структурой) и слой хранения данных (PostgreSQL и MongoDB), дополненный CLI-вызовом Python-скриптов модуля машинного обучения.

2.3.1 Общая схема архитектуры

Клиентская часть реализована как одностраничное приложение на Vue 3 с использованием Vite и TypeScript. Связь с сервером обеспечивает обёртка axios в файле src/api/http.ts: интерцептор автоматически добавляет в каждый запрос заголовки X-User-Id и Authorization: Bearer JWT. Серверная часть на Go обслуживает HTTP-запросы через Gin, выполняет бизнес-логику в слое сервисов и обращается к PostgreSQL через GORM и к MongoDB через mongo-go-driver. Для построения персональных рекомендаций сервер по требованию запускает Python-скрипты модуля ml через exec.Command, передавая

параметры через аргументы и получая результат в виде JSON через stdout. Общая архитектурная схема системы показана на Рисунке 2.3.

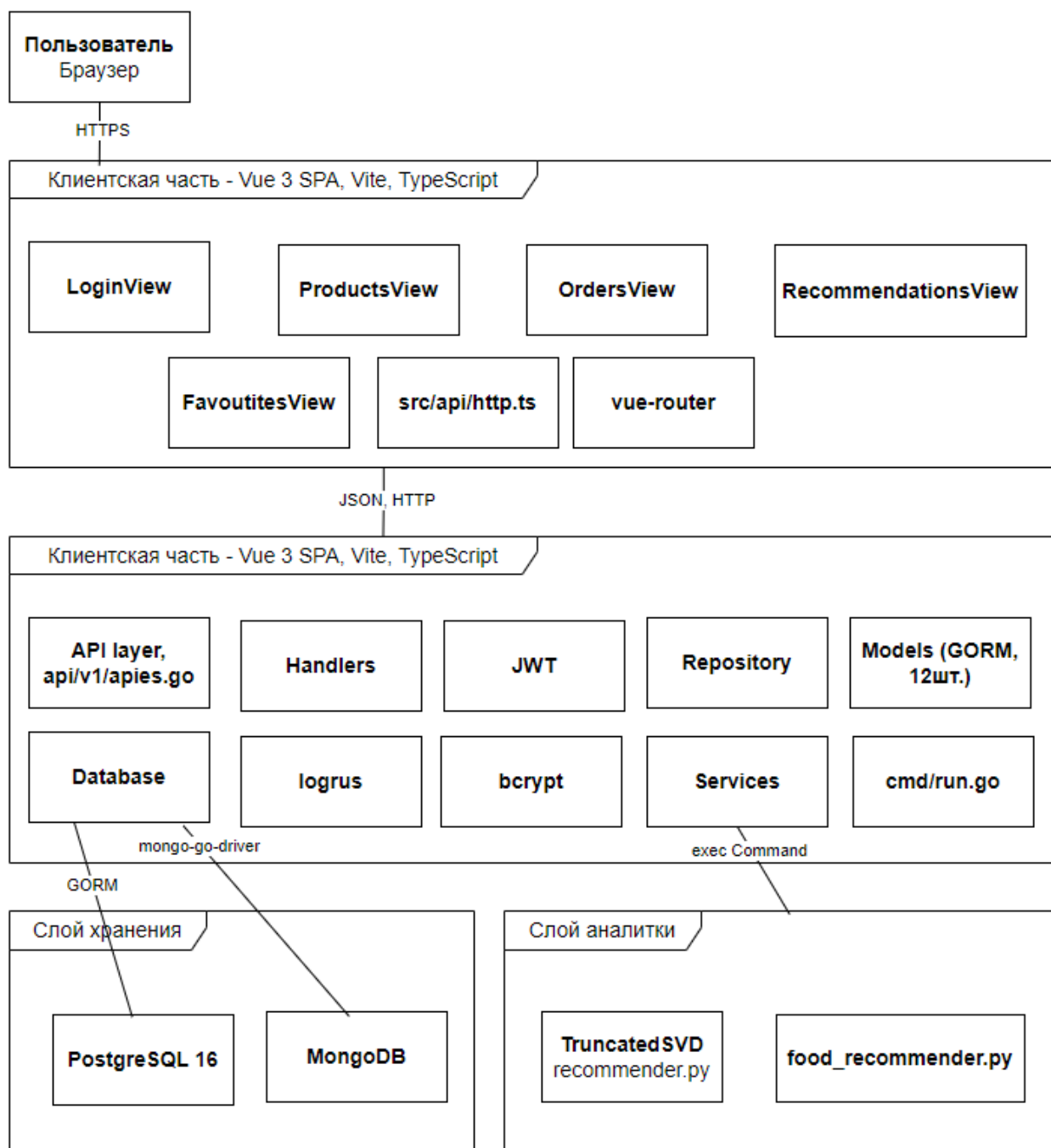


Рисунок 2.3 — Общая архитектурная схема информационной системы

Архитектурная схема демонстрирует чёткое разделение ответственности между уровнями: клиент отвечает за визуализацию и локальную работу с корзиной; серверный модуль на Go – за бизнес-логику, авторизацию и оркестрацию обращений к хранилищам; PostgreSQL и MongoDB — за долговременное хранение структурированных и бинарных

данных соответственно; Python-скрипты – за тяжёлые аналитические вычисления, выполняемые по требованию. Такая декомпозиция позволяет независимо развивать каждый из уровней.

2.3.2 Структура данных

В PostgreSQL развёрнуто 12 сущностей (моделей GORM, файлы `internal/models/*.go`):

- User – учётная запись пользователя: name, role (admin/user/cashier), password_hash (bcrypt), gender (male/female), hired_at;
- Product – товар каталога: name, category_id (внешний ключ), manufacturer_id (внешний ключ), barcode (UNIQUE), default_price, calories_kcal, protein_g, fat_g, carbs_g, created_at;
- Category – категория товара: name;
- Manufacturer – производитель: name, country, contact_info (jsonb);
- Order – заказ: cashier_id (внешний ключ на User), created_at, total_amount;
- OrderItem – позиция заказа: order_id, product_id, quantity, price_per_unit, discount;
- Dish – блюдо: name;
- DishProduct – связь блюда с конкретным товаром: dish_id, product_id, quantity, price_per_unit, discount;
- DishCategoryRequirement – требование блюда к ингредиенту, заданному категорией: dish_id, category_id (необязательно), ingredient_name, quantity, note;
- FavouriteProduct – избранный товар пользователя: user_id, product_id;
- FavouriteDish – избранное блюдо пользователя: user_id, dish_id.

Все сущности приведены к третьей нормальной форме [12]. Основные связи: User имеет связь один-ко-многим с Order; Order – с OrderItem; Product –

с OrderItem; Category – с Product; Manufacturer – с Product; Dish – с DishProduct и DishCategoryRequirement; Category – с DishCategoryRequirement; User – с FavouriteProduct и FavouriteDish.

В MongoDB (база products_db, коллекция products) хранятся изображения товаров в виде документов следующей структуры: product_id: int64, content_type: string, data: BinData, где product_id – идентификатор товара в PostgreSQL, content_type – MIME-тип бинарного содержимого, data – байты изображения.

Сущности базы данных и связи между ними представлены на Рисунке 2.4.

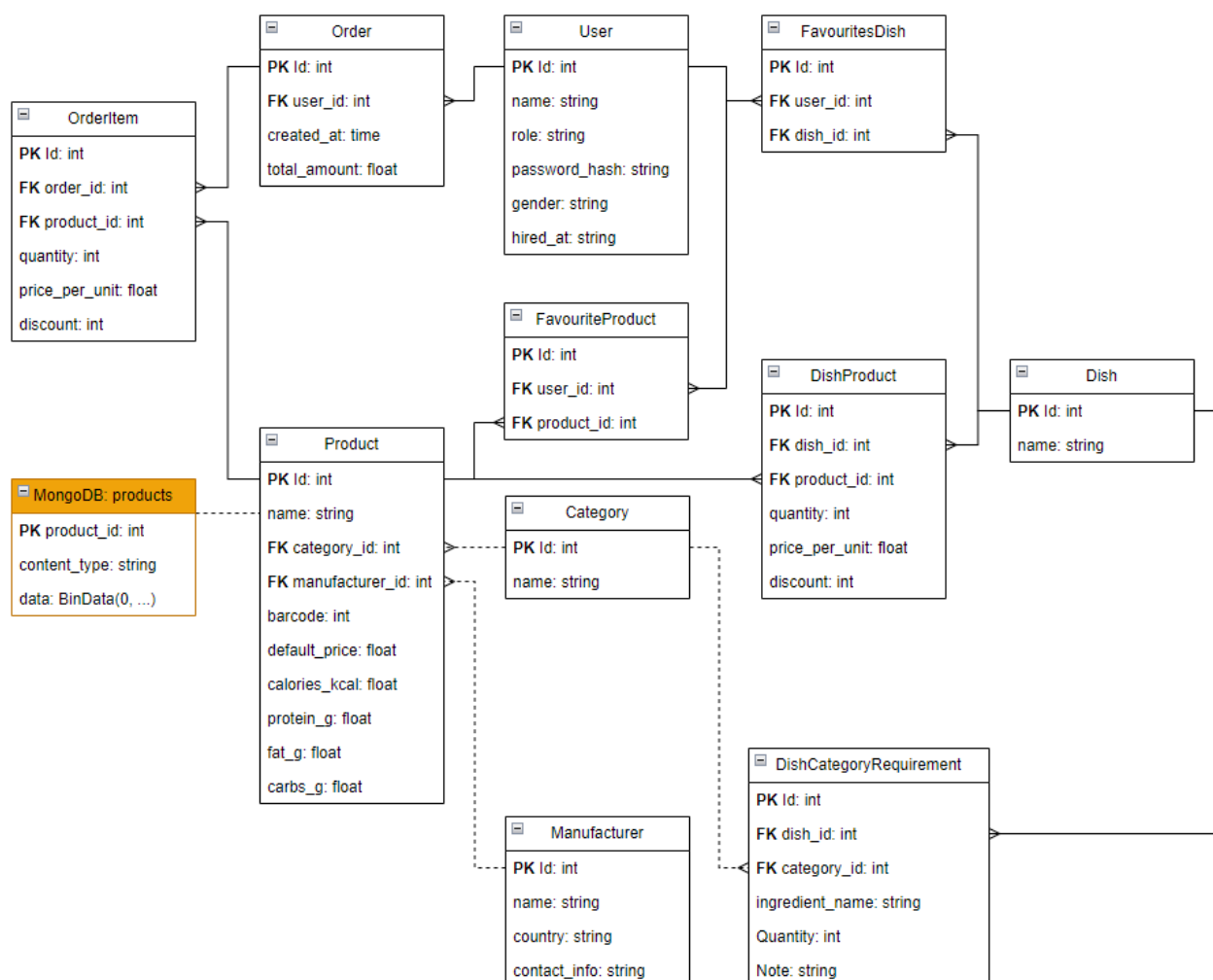


Рисунок 2.4 — ER-диаграмма базы данных

Связь между хранилищами реализована через product_id, а не через идентификатор изображения, хранимый в PostgreSQL. Такой подход

позволяет полностью отделить вопросы загрузки изображений от схемы товара (при отсутствии изображения соответствующего товара эндпоинт GET /v1/products/:id/image возвращает HTTP 404 без обращения к PostgreSQL).

Представленная модель данных охватывает все основные сценарии работы системы: ведение каталога товаров и блюд, оформление заказов, фиксацию избранных позиций для построения рекомендаций и хранение бинарных изображений в отдельной коллекции MongoDB. Использование третьей нормальной формы исключает дублирование данных и упрощает поддержку целостности [13].

2.3.3 Логика взаимодействия компонентов

Жизненный цикл запроса на формирование рекомендации блюд по корзине: клиент Vue 3 (OrdersView.vue) собирает локальную корзину из localStorage и отправляет её POST-запросом на /v1/users/:id/meals/from-cart; Go-хендлер RecommendationHandler.GetMealsFromCart валидирует JWT и список товаров, передаёт корзину в RecommendationService.DishesFromCart; сервис подгружает все блюда с предзагрузкой состава, для каждого блюда выполняет функцию scoreDishByCategories (или scoreDishByProducts, если состав задан конкретными товарами), считает доли совпавших ингредиентов и нутриентный скор, формирует список недостающих ингредиентов с подсказками; результат возвращается клиенту.

Жизненный цикл запроса на персональные рекомендации товаров: клиент обращается к эндпоинту /v1/users/:id/recommendations/products; Go-сервис пытается выполнить расчёт через Python-скрипт ml/food_recommender.py (вызов exec.Command, передача строки подключения к PostgreSQL через флаг --db-url, параметризация по --user-id и --k); при отказе Python (нет интерпретатора, нет данных) применяется fallback на простую контент-ориентированную эвристику в Go (RecommendationService.RecommendProducts), вычисляющую скор кандидатов на основе совпадения категорий и производителей с избранными

товарами пользователя. Диаграмма последовательности процесса подбора блюд по корзине приведена на Рисунке 2.5.

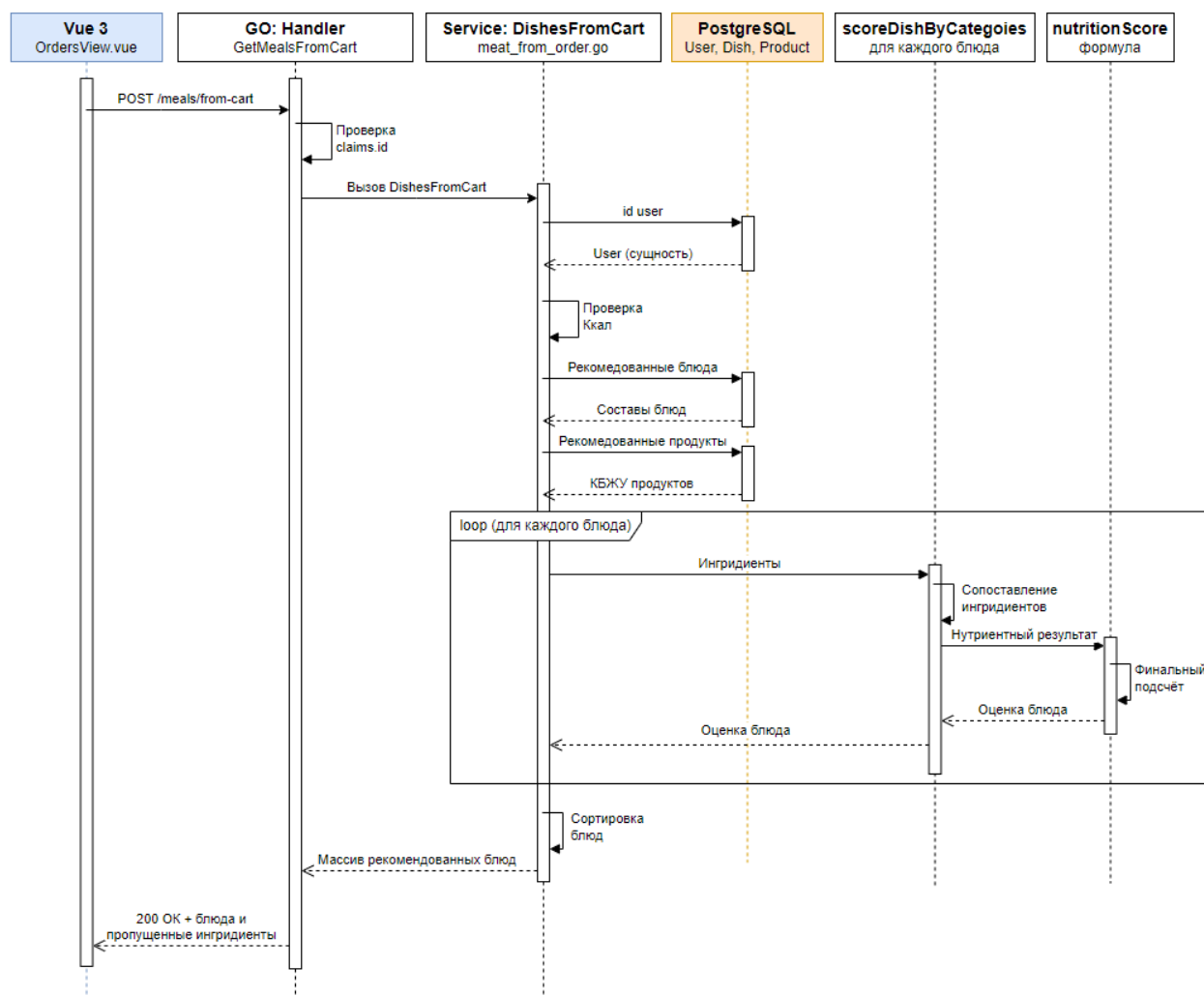


Рисунок 2.5 — Диаграмма последовательности процесса подбора блюд по корзине

Из диаграммы последовательности видно, что критический путь запроса содержит ровно одно обращение к PostgreSQL и один проход по справочнику блюд с предзагрузкой состава. Это позволяет уложиться в целевые 200 мс отклика и сохраняет линейную зависимость времени работы от размера каталога блюд.

2.3.4 Многослойная структура серверной части

Go-приложение организовано по принципу многослойной архитектуры:

- слой API (internal/api/v1/apies.go) – регистрация маршрутов Gin, middleware CORS, JWT и adminOnly;

- слой Handlers (internal/handlers/*) – приём HTTP-запросов, валидация, формирование JSON-ответов (auth.go, product.go, category.go, manufacturer.go, dishes.go, order.go, favourite.go, recommendation.go, admin_users.go, product_image.go);
- слой Services (internal/services/*) – бизнес-логика: order_service.go, product_service.go, recommendation_service.go, meal_from_order.go, python_recommender_service.go;
- слой Models (internal/models/*) – структуры данных и объекты передачи данных для моделей GORM;
- слой Database (internal/database/*) – конфигурация и инициализация соединений с PostgreSQL и MongoDB;
- слой Repository (internal/repository/product_image.go) – низкоуровневые операции с MongoDB-коллекцией изображений.

Компонентная диаграмма серверной части (шесть слоёв) представлена на Рисунке 2.6. Сплошные линии – отношение «реализует» или «содержит», а пунктирные – «использует».

Шестислойная архитектура серверной части обеспечивает чёткое разделение ответственности и упрощает дальнейшее развитие системы: изменения в способе хранения изображений (например, переход на GridFS) затронут только слой Repository; изменения в алгоритмах рекомендаций – только слой Services; изменения в маршрутах API – только слой API.

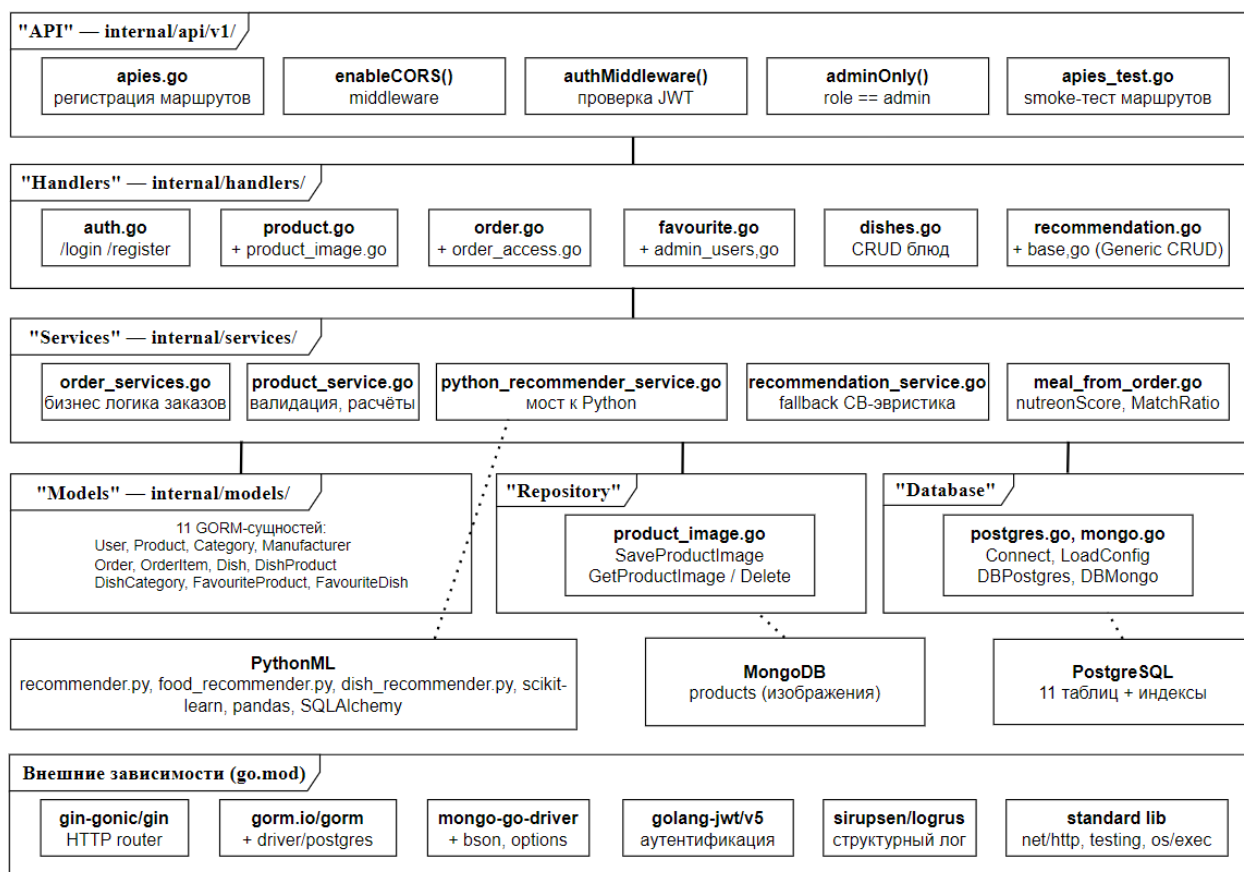


Рисунок 2.6 — Компонентная диаграмма серверной части

Это удовлетворяет требованию архитектурной устойчивости, заложенному в нефункциональных требованиях [14].

2.4 Обоснование выбора технологического стека

Выбор языков, фреймворков и систем управления базами данных продиктован архитектурными задачами системы: эффективная обработка параллельных HTTP-запросов, поддержка вычислений в режиме реального времени для расчёта нутриентного баланса, гибкое хранение разнородной информации (структурированных и бинарных данных) и отзывчивый клиентский интерфейс.

2.4.1 Серверная часть: язык Go

В качестве языка серверной части выбран Go (Golang) [15]. Альтернативами рассматривались Python (Django и FastAPI), Java (Spring Boot)

и Node.js. Python ограничен глобальной блокировкой интерпретатора при параллельных вычислениях; Java требует значительных ресурсов виртуальной машины и характеризуется большим объёмом шаблонного кода; Node.js эффективен для операций ввода-вывода, но менее подходит для CPU-интенсивных задач. Go компилируется в нативный бинарный файл, обеспечивает строгую статическую типизацию и поддерживает легковесную модель конкурентности (горутины и каналы) [16], что позволяет одновременно обслуживать множество запросов на расчёт рекомендаций. В проекте используются: фреймворк Gin [17] для HTTP-маршрутизации, GORM [18] для работы с PostgreSQL, mongo-go-driver для MongoDB, golang-jwt/v5 для работы с JWT-токенами, golang.org/x/crypto/bcrypt для хеширования паролей, sirupsen/logrus для структурного логирования.

2.4.2 Клиентская часть: Vue 3

Для клиентской части выбран фреймворк Vue 3 [19] (версия 3.5) с инструментом сборки Vite [20] (версия 8) и TypeScript (версия 5.9) [21]. По сравнению с React фреймворк Vue 3 предоставляет более согласованную экосистему «из коробки» (vue-router, реактивность на основе Composition API) и не требует подбора сторонних библиотек. По сравнению с Angular фреймворк Vue более лёгкий и менее структурно навязчивый, что упрощает реализацию компактного одностраничного приложения. Управление состоянием намеренно реализовано без Pinia и Vuex: контекст пользователя и корзина хранятся в localStorage, а реактивные обновления распространяются через нативное событие cart-changed, что снижает зависимости и ускоряет загрузку.

2.4.3 Хранилища данных

PostgreSQL [22] выбрана как реляционная система управления базами данных, обеспечивающая ACID-гарантии, развитую систему индексов и

поддержку сложных аналитических запросов (соединения таблиц при формировании рекомендаций), а также тип `jsonb` (используется в `Manufacturer.ContactInfo`). MongoDB [23] используется как хранилище неструктурированного бинарного контента – изображений товаров. Документоориентированная модель и динамическая схема позволяют хранить изображение целиком как поле `data` с метаданными `product_id` и `content_type`, без обязательной нормализации [24].

2.4.4 Модуль машинного обучения: Python

Аналитический слой реализован на Python с использованием библиотек `scikit-learn` [25] (`TruncatedSVD`), `pandas` [26] и `SQLAlchemy` [27]. Эти библиотеки предоставляют зрелые реализации матричного разложения и удобные средства работы с данными в формате `DataFrame`. Интеграция с Go-сервером выполнена через интерфейс командной строки: Go-сервис запускает Python-скрипт командой `exec.Command`, передаёт параметры (строку подключения, идентификатор пользователя, число рекомендаций) аргументами командной строки и получает результат в виде JSON через стандартный вывод. Такой подход проще, чем поднимать отдельный HTTP-микросервис, и достаточен для текущего масштаба нагрузки. Сравнение выбранного технологического стека с альтернативами по ключевым критериям приведено в Таблице 2.2.

Таблица 2.2 — Сравнение технологических стеков по ключевым критериям

Критерий	Выбранный стек: Go + Vue 3	Альтернатива: Python + React	Альтернатива: Java + Angular
Скорость выполнения CPU	Высокая (компиляция в нативный код)	Средняя (интерпретатор)	Высокая (JIT-компиляция JVM)
Модель конкурентности	Отличная (горутины, каналы)	Ограниченная (GIL)	Хорошая (потoki JVM)
Потребление оперативной памяти	Минимальное (≈ 30–50 МБ)	Среднее (≈ 100–150 МБ)	Высокое (≈ 300+ МБ на JVM)

Продолжение Таблицы 2.2

Целостность экосистемы клиентской части	Высокая (Vue 3 самодостаточен: vue-router, Composition API)	Низкая (фрагментация React-библиотек)	Максимальная (монолит Angular)
Поддержка гибридного хранения	Да (PostgreSQL + MongoDB через драйверы)	Да	Да
Объём шаблонного кода	Малый	Малый	Большой (Spring boilerplate)
Соответствие задачам ВКР	Полное (выбран)	Частичное	Избыточное по ресурсам

Из сводной таблицы следует, что связка Go и Vue 3 в сочетании с гибридным хранилищем PostgreSQL и MongoDB обеспечивает требуемый баланс между производительностью, ресурсоёмкостью и гибкостью схемы данных, превосходя альтернативные стеки по большинству ключевых критериев.

Подводя итог второй главе, можно сделать следующие выводы:

- выполнено моделирование бизнес-процессов в нотации BPMN: текущий процесс “As-Is” демонстрирует пассивную роль информационной системы, а целевой процесс “To-Be” включает контур обратной связи и автоматический подбор блюд по составу корзины;
- сформулированы функциональные требования FR-01 – FR-09, охватывающие регистрацию и аутентификацию, управление каталогом, хранение изображений, формирование заказа, подбор блюд по корзине и заказу, персональные рекомендации, избранное и администрирование;
- определены нефункциональные требования: производительность (отклик до 200 мс для базовых операций и до 500 мс для гибридных рекомендаций), масштабируемость, надёжность на основе ACID-гарантий PostgreSQL, безопасность с использованием bcrypt и JWT, адаптивность интерфейса;
- разработана архитектура системы по клиент-серверной модели с гибридным хранилищем (PostgreSQL и MongoDB) и многослойной

структурой серверной части на Go (шесть слоёв: API, Handlers, Services, Models, Database, Repository); связь хранилищ реализована через идентификатор товара `product_id`;

- обоснован выбор технологического стека: язык Go для серверной части за счёт конкурентной модели горутин, фреймворк Vue 3 для клиентской части за счёт согласованной экосистемы, PostgreSQL и MongoDB для гибридного хранения и Python для аналитического слоя машинного обучения.

Сформулированные требования и спроектированная архитектура служат непосредственной основой для технической реализации системы, которой посвящена глава 3.

Принципиальной особенностью спроектированного решения является сочетание реляционного и документоориентированного подходов к хранению данных. Реляционная модель PostgreSQL обеспечивает целостность и согласованность для тех данных, к которым применимы строгие нормализационные требования (профили пользователей, заказы, спецификации блюд), тогда как документоориентированная модель MongoDB снимает с реляционной базы нагрузку, связанную с хранением и выдачей крупных бинарных объектов. Связь между хранилищами через общий идентификатор товара исключает необходимость поддерживать дополнительный индекс изображений в PostgreSQL и упрощает каскадную чистку данных при удалении товара.

Многослойная организация серверной части на Go обоснована не только соображениями архитектурной чистоты, но и практическими задачами развития системы. Выделенный слой Repository изолирует код работы с MongoDB и в дальнейшем позволит безболезненно перейти к GridFS либо к внешнему объектному хранилищу типа S3. Выделенный слой Services концентрирует бизнес-логику и облегчает её повторное использование как из HTTP-обработчиков, так и из возможных в будущем фоновых заданий.

Обобщённый CRUD-хендлер на дженериках сокращает объём кода и снижает риск рассогласования поведения справочных эндпоинтов.

Выбор интеграции с Python через интерфейс командной строки, а не через отдельный HTTP-микросервис, является осознанным проектным решением и отражает текущий масштаб системы: при характерной нагрузке в десятки одновременных пользователей накладные расходы на запуск Python-процесса оказываются ниже, чем затраты на поддержание отдельной инфраструктуры. Однако спроектированная архитектура сохраняет возможность последующей замены этого механизма на полноценный микросервис без изменения публичного API системы — соответствующее направление развития указано в заключении.

3 ТЕХНИЧЕСКАЯ РЕАЛИЗАЦИЯ СИСТЕМЫ

3.1 Разработка структуры базы данных

3.1.1 Гибридная модель хранения

Хранилище данных реализовано как комбинация двух систем управления базами данных. PostgreSQL отвечает за транзакционный слой: учётные записи, каталог товаров с детальными нутриентными характеристиками, заказы, блюда, требования блюд к ингредиентам и избранные позиции. MongoDB используется как хранилище бинарного контента – изображений товаров. Связь между хранилищами реализована не через хранимое в PostgreSQL поле-указатель, а через идентификатор товара `product_id`, фигурирующий как поле документа MongoDB. При отсутствии изображения соответствующего товара эндпоинт `GET /v1/products/:id/image` возвращает HTTP 404 без обращения к PostgreSQL.

3.1.2 Логическое проектирование

Структура PostgreSQL приведена к третьей нормальной форме. Список сущностей и связей дан в пункте 2.3.2. Создание схемы и поддержание её актуальности обеспечивает функция `database.CreateObjDB`, вызывающая `gorm.AutoMigrate` для всех 12 моделей при старте сервиса (`cmd/run.go`). Это гарантирует согласованность между структурами Go и схемой базы данных без необходимости поддерживать отдельные SQL-миграции.

3.1.3 Детальная спецификация ключевых таблиц

Спецификация сущности «Товар» приведена в Таблице 3.1.

Таблица 3.1 — Спецификация сущности «Товар» (Product)

Поле	Тип и ограничения	Описание
id	BIGSERIAL, PK	Уникальный идентификатор товара
name	TEXT, NOT NULL	Наименование товара
category_id	BIGINT, FK на Category	Категория товара (необязательно)
manufacturer_id	BIGINT, FK на Manufacturer	Производитель (необязательно)
barcode	TEXT, UNIQUE	Штрихкод товара, уникален
default_price	NUMERIC(12,2), NOT NULL	Базовая цена единицы товара
calories_kcal	NUMERIC(10,2)	Энергетическая ценность на единицу товара
protein_g	NUMERIC(10,2)	Содержание белков, г
fat_g	NUMERIC(10,2)	Содержание жиров, г
carbs_g	NUMERIC(10,2)	Содержание углеводов, г
created_at	TIMESTAMP TZ	Дата создания записи

Для вычисления суммарного вектора КБЖУ корзины применяется простое суммирование по позициям заказа: суммарные значения калорийности, белков, жиров и углеводов вычисляются как сумма произведений количества единиц товара в корзине на соответствующие нутриентные характеристики каждого товара. Такая агрегация выполняется и на клиенте (для отображения текущей сводки), и на сервере (при расчёте нутриентного счёта блюд).

Спецификация сущности «Блюдо» и связанных с ней таблиц приведена в Таблице 3.2.

Таблица 3.2 — Спецификация сущности «Блюдо» и связанных таблиц

Сущность	Ключевые поля	Назначение
Dish	id, name	Описание блюда (имя). Состав задаётся в DishProduct и/или DishCategoryRequirement
DishProduct	dish_id, product_id, quantity, price_per_unit, discount	Состав блюда, заданный конкретными товарами (жёсткая привязка)

Продолжение Таблицы 3.2

DishCategoryRequirement	dish_id, category_id (опц.), ingredient_name, quantity, note	Требование блюда к ингредиенту, выраженное через категорию или текстовое имя ингредиента (мягкая привязка)
-------------------------	--	--

Двойная схема описания состава блюда – жёсткая через DishProduct и мягкая через DishCategoryRequirement – является особенностью реализации. Она позволяет администратору вносить как классические рецепты с конкретными ингредиентами, так и обобщённые шаблоны вида «нужно <Категория> в количестве N единиц».

3.1.4 Документоориентированное хранилище изображений

В MongoDB используется коллекция “products” базы products_db (см. константы productImagesCollection и MongoDBDatabaseName в коде). Документ имеет следующую структуру:

{ product_id: 1024, content_type: “image/jpeg”, data: BinData(0, "...") }, где product_id – тот же идентификатор, что и в PostgreSQL; content_type – MIME-тип бинарного содержимого; data – байты изображения.

Сохранение и удаление изображений выполняется в репозитории internal/repository/product_image.go: функция SaveProductImage использует ReplaceOne с upsert=true по фильтру product_id, что гарантирует наличие не более одного документа на товар; GetProductImage читает документ через FindOne; DeleteProductImage удаляет документ по тому же фильтру. На уровне HTTP-обработчика handlers/product_image.go при выдаче изображения проставляется content_type из документа MongoDB.

3.1.5 Стратегия индексации

GORM на этапе авто-миграции создаёт первичные ключи и индексы по тегам: индексы по category_id, manufacturer_id и другим внешним ключам моделей помечены тегом gorm:"index". Уникальные ограничения: barcode у

Product (тег `gorm:"unique"`). В MongoDB используется композитный фильтр по `product_id`.

Целевое время отклика для типовых операций (выборка списка товаров, добавление в корзину, получение деталей) – до 100 мс на стороне сервера, что подтверждается результатами нагрузочного тестирования (см. главу 4).

3.2 Реализация серверной части

3.2.1 Точка входа и инициализация

Точка входа сервиса – `main.go`, вызывающий `cmd.Run()` (`cmd/run.go`). На старте выполняются: инициализация логгера (`utils.InitLogger`), загрузка переменных окружения через `godotenv`, подключение к PostgreSQL (`database.Connect`) и к MongoDB (`database.ConnectMongoDB`), авто-миграция 12 GORM-моделей, опциональное создание пользователя-администратора по переменным `ADMIN_NAME` и `ADMIN_PASSWORD`, и запуск HTTP-сервера через `v1.Apies()`. Обработка сигналов `SIGINT` и `SIGTERM` с корректным завершением реализована в файле `api/v1/apies.go`.

3.2.2 Регистрация маршрутов и middleware

Все маршруты группируются в файле `internal/api/v1/apies.go` (полный листинг приведён в Приложении А). Корневые маршруты `POST /login` и `POST /register` – публичные. Защищённые маршруты группируются под префиксом `/v1` и проходят через `authMiddleware`, который читает JWT-токен из заголовка `Authorization: Bearer` и сохраняет идентификатор, имя и роль пользователя в Gin-контексте. Для операций изменения каталога дополнительно применяется `adminOnly`, требующий роль `admin`. Полный перечень основных маршрутов:

- `/login` и `/register` – аутентификация и регистрация;
- `/v1/users`, `/v1/users/:id`, `/v1/users/full`, `/v1/users/:id/full` – управление пользователями (создание, изменение и удаление только для `admin`);

- `/v1/users/:id/meals/from-order/:orderId` – подбор блюд по ранее оформленному заказу;
- `/v1/users/:id/meals/from-cart` – подбор блюд по текущей корзине;
- `/v1/users/:id/recommendations/products`,
`/v1/users/:id/recommendations/dishes`, `/v1/users/:id/recommendations/final` – персональные рекомендации;
- `/v1/categories` и `/v1/manufacturers` – справочники (чтение публично, изменения только для admin);
- `/v1/products`, `/v1/products/:id`, `/v1/products/:id/image` – каталог товаров и изображения;
- `/v1/dishes`, `/v1/dishes/:id`, `/v1/dishes/:id/products`,
`/v1/dishes/:id/category-requirements` – справочник блюд и их составов;
- `/v1/orders` и `/v1/order-items` – заказы и их позиции;
- `/v1/favourites/products` и `/v1/favourites/product/:productId` – избранные товары.

3.2.3 Обобщённый CRUD-хендлер

Чтобы избежать дублирования кода операций над разными моделями, в проекте использован обобщённый хендлер `BaseHandler[T]` с методами `GetAll`, `GetByID`, `Create`, `Update`, `Delete` (файл `internal/handlers/base.go`). Он параметризуется типом `T` и принимает указатель на `gorm.DB` через структуру. Конкретные хендлеры (категорий, производителей, пользователей, позиций заказа) получают простым инстанцированием `BaseHandler[Models.X]`. Это позволило быстро поднять полный набор операций над основными справочниками.

3.2.4 Аутентификация и авторизация

Реализация аутентификации сосредоточена в файле `internal/handlers/auth.go`. При регистрации (POST `/register`) пользователь

передаёт name, password и gender (валидируется как “male” или “female”). Пароль хешируется через bcrypt.GenerateFromPassword [28] и сохраняется в поле password_hash модели User. При входе (POST /login) сервер ищет пользователя по name, проверяет хеш через bcrypt.CompareHashAndPassword и при успехе генерирует токен JSON Web Token (JWT, формат описан в RFC 7519 [29]) с полями claims user_id, name, role, exp (срок действия – 24 часа). Секрет JWT берётся из переменной окружения JWT_SECRET (по умолчанию – dev-секрет с предупреждением). На стороне клиента токен сохраняется в localStorage под ключом authToken и прикрепляется к каждому запросу через axios-интерцептор.

3.2.5 Работа с базами данных

PostgreSQL подключается через gorm.io/driver/postgres (internal/database/postgres.go). Параметры берутся либо из переменной POSTGRES_DSN, либо собираются из переменных POSTGRES_HOST, POSTGRES_PORT, POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB. Глобальное соединение хранится в database.DbPostgres. MongoDB подключается через go.mongodb.org/mongo-driver (internal/database/mongo.go); параметры – MONGO_URI, MONGO_HOST, MONGO_PORT, MONGO_DB (по умолчанию products_db). Глобальный клиент – database.DbMongo. Закрытие обоих соединений происходит при штатном завершении процесса.

3.2.6 Управление каталогом, заказами и блюдами

Хендлеры в internal/handlers/{product, category, manufacturer, dishes, order, favourite}.go реализуют валидацию и сохранение через GORM с предзагрузкой связанных сущностей (механизм Preload). Загрузка изображения товара (handlers/product_image.go) принимает multipart-форму, читает файловую часть и сохраняет её в MongoDB через

repository.SaveProductImage; выдача — обратная операция. Удаление товара (функция deleteProductAndImage в apies.go) каскадно сначала вызывает repository.DeleteProductImage, затем удаляет запись Product.

3.3 Реализация клиентской части

3.3.1 Структура одностраничного приложения

Клиентская часть представляет собой одностраничное приложение, организованное по модульному принципу. Корневые файлы: src/main.ts (инициализация Vue), src/App.vue (корневой компонент с навигацией), src/style.css (общие стили), src/router/index.ts (маршрутизация). Маршруты заданы статически и включают: /, /register, /products, /favourites, /orders, /categories, /manufacturers, /dishes, /recommendations, /users.

3.3.2 Слой взаимодействия с серверным API

Файл src/api/http.ts создаёт экземпляр axios [30] с базовым адресом http://localhost:8080. На каждый запрос интерцептор автоматически добавляет заголовки X-User-Id (текущий пользователь из localStorage) и Authorization: Bearer JWT, если соответствующие значения присутствуют. Для запроса изображения товара экспортируется отдельная вспомогательная функция productImageUrl(id). Все типы объектов передачи данных объявлены в этом же файле, что обеспечивает строгую типизацию запросов и ответов на стороне TypeScript.

3.3.3 Представления (views)

- LoginView.vue, RegisterView.vue – формы аутентификации и регистрации, сохранение JWT-токена и профиля в localStorage;

- `ProductsView.vue` – каталог товаров: поиск, фильтрация по категориям, сортировка, добавление в корзину и в избранное, для пользователя-admin – формы изменения каталога;
- `OrdersView.vue` – корзина и история заказов: чтение и сохранение `cartItems` в `localStorage`, оформление заказа через серию `POST /v1/orders` и `/v1/order-items`, отображение текущей сводной калорийности корзины;
- `CategoriesView.vue`, `ManufacturersView.vue`, `DishesView.vue`, `UsersView.vue` – справочники с операциями создания, изменения и удаления (для admin);
- `FavouritesView.vue` – управление избранными товарами через эндпоинты `/v1/favourites/*`;
- `RecommendationsView.vue` – отображение результатов работы рекомендательного модуля: список товаров с разбивкой по компонентам сора (КБЖУ, история, блюда, давность), список рекомендованных блюд и метрика `precision@5`, возвращаемая Python-скриптом.

3.3.4 Управление состоянием и реактивная корзина

Управление состоянием реализовано без сторонних библиотек: `Pinia` и `Vuex` намеренно не используются. Контекст пользователя (`currentUserId`, `currentUserName`, `currentUserRole`, `currentUserGender`, `authToken`) и текущая корзина (`cartItems`) сохраняются в `localStorage`. Реактивность интерфейса при изменении корзины обеспечивается событийной моделью: при каждом изменении корзины (добавление, удаление, очистка) вызывается `window.dispatchEvent` с событием `cart-changed`. Все компоненты, отображающие сводные показатели (текущая КБЖУ, число позиций), подписываются на это событие в хуке `onMounted` и автоматически пересчитывают данные. Схема взаимодействия компонентов клиентского приложения приведена на Рисунке 3.1.

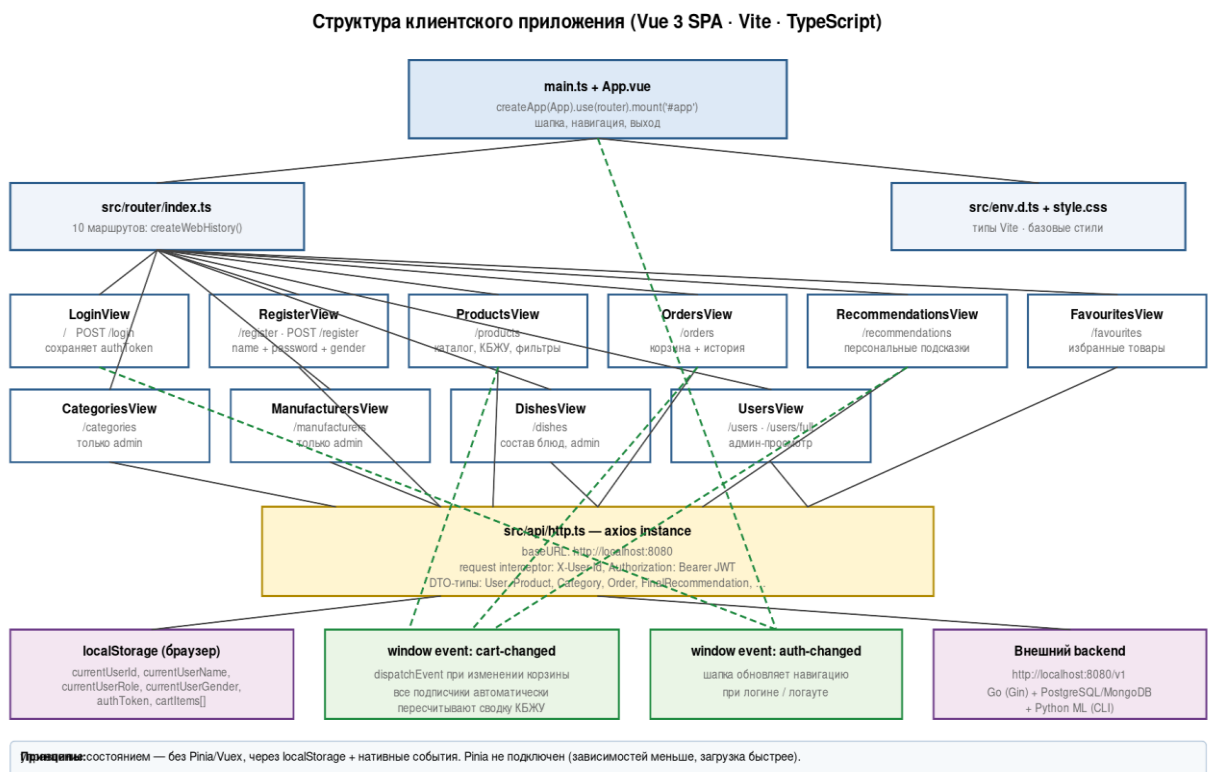


Рисунок 3.1 — Схема взаимодействия компонентов Vue 3 SPA

Применение событийной модели вместо централизованного менеджера состояния делает клиентскую часть более лёгкой и предсказуемой: каждый компонент явно подписывается на интересующие его события и не зависит от глобального дерева состояния. Это снижает риск утечек подписок и упрощает модульное тестирование отдельных представлений.

3.3.5 Загрузка изображений товаров

В административном режиме ProductsView.vue позволяет загружать изображения товаров. Файл отправляется как multipart-форма на POST /v1/products/:id/image; сервер сохраняет бинарное содержимое в MongoDB (см. пункт 3.1.4). При отображении карточек клиент использует функцию productImageUrl(id) (GET /v1/products/:id/image); если изображение не найдено (HTTP 404), отображается заглушка «нет фото».

3.4 Реализация рекомендательного модуля

Рекомендательный модуль построен по двухслойной схеме: эвристический слой на Go (быстрый, синхронный, работает на каждый запрос корзины) и аналитический слой на Python (запускается по требованию через интерфейс командной строки). Такой подход совмещает преимущества компилируемой нативной обработки на стороне сервера и зрелую экосистему `scikit-learn` для матричных вычислений [31].

3.4.1 Эвристический слой: подбор блюд по корзине

Сервис `RecommendationService.DishesFromCart` (файл `internal/services/meal_from_order.go`) принимает корзину в виде словаря, отображающего идентификатор товара в количество единиц. Он подгружает все блюда из базы данных с предзагрузкой `Products` (жёсткая привязка) и `CategoryRequirements` (мягкая привязка). Для каждого блюда вызывается функция `scoreDishByCategories` или `scoreDishByProducts`.

В случае мягкой привязки выполняется нормализация имени ингредиента – функция `normalizeIngredientKey`: имя приводится к нижнему регистру, удаляются упаковочные токены вида «1 кг», «500 г», «10 шт» (регулярное выражение `packageTokenRe`), знаки пунктуации заменяются пробелами. По нормализованному ключу строится индекс товаров в корзине, и для каждого требования блюда фиксируется, найден ли подходящий ингредиент. Если в корзине нет товара под требование, КБЖУ ингредиента вычисляется как среднее по всем товарам соответствующей категории в каталоге.

По результатам сопоставления для каждого блюда вычисляются метрики: `matched_items` и `required_items` (число совпавших и требуемых ингредиентов), `MatchRatio = matched / required`, `Makeable` (можно ли собрать блюдо полностью), `TotalKcal`, `Protein`, `Fat`, `Carbs` (расчётное КБЖУ блюда), и

список Missing — недостающие ингредиенты с подсказками-товарами из каталога (Suggestions: до 15 товаров, отсортированных по возрастанию цены).

Нутриентный скор блюда вычисляется функцией nutritionScore по формуле:

$$score = 50 \cdot balance + 50 \cdot calFit \quad (3.1)$$

где значения balance и calFit определяются как:

$$balance = 1 - (|\hat{p} - 0,25| + |f - 0,30| + |\hat{c} - 0,45|) \quad (3.2)$$

$$calFit = 1 - \min(1, |kcal - mealTarget| / mealTarget) \quad (3.3)$$

Здесь $\hat{p}$, $\hat{f}$, $\hat{c}$ — доли калорий, приходящиеся на белки, жиры и углеводы в блюде (например, $\hat{p} = (4 \cdot protein) / (4 \cdot protein + 9 \cdot fat + 4 \cdot carbs)$); mealTarget — целевая калорийность приёма пищи (833,3 ккал для мужчин и 666,7 ккал для женщин). Слагаемое balance оценивает близость соотношения белков, жиров и углеводов к рекомендуемому 25, 30 и 45 процентов, слагаемое calFit — близость общей калорийности блюда к целевой. Оба компонента принимают значения в диапазоне от 0 до 1, итоговое значение score — в диапазоне от 0 до 100.

Сортировка финального списка блюд (функция limitAndSortMeals) идёт по убыванию приоритета: сначала по MatchRatio, затем по числу совпавших ингредиентов, затем по признаку собираемости, в последнюю очередь по nutritionScore. Это поднимает в верх выдачи блюда, которые пользователь почти может приготовить из текущей корзины, а среди равных по совпадению — наиболее сбалансированные по КБЖУ.

Аналогичный сервис DishesFromOrder работает не с корзиной, а с готовым заказом из базы данных (эндпоинт /v1/users/:id/meals/from-order/:orderId): он загружает заказ, проверяет принадлежность пользователю (иначе возвращает ошибку ErrOrderAccess), собирает доступные товары из его OrderItem и далее проходит ту же логику скоринга.

3.4.2 Аналитический слой: персональные рекомендации

Аналитический слой представлен тремя Python-скриптами в директории `ml`:

- `recommender.py` – для рекомендации блюд через матричное разложение [32]. Строит разреженную матрицу взаимодействий «пользователь – блюдо» из таблиц `orders` и `order_items` (через связку `DishProduct`), применяет `TruncatedSVD` из `scikit-learn` [33] (`n_components = 24`) и возвращает топ-К блюд для целевого пользователя, ранжированных по dot-произведению скрытых факторов;
- `food_recommender.py` – гибридный рекомендатель отдельных товаров. Считает четыре компонента сора для каждого кандидата: СВ (контент-ориентированная близость КБЖУ товара к целевому профилю пользователя) [34], CF (коллаборативная фильтрация по истории заказов), `meal` (соответствие товара блюдам, рекомендованным первым скриптом), `recency` (штраф за недавние покупки) [35]. Итоговый скор вычисляется как сумма произведений весов и компонент;
- `dish_recommender.py` — расширенный вариант с фильтром по избранному и буст-коэффициентами для активных заказов.

Итоговая формула гибридного сора в `food_recommender.py` имеет вид:

$$final = w_{cb} \cdot CB + w_{cf} \cdot CF + w_{meal} \cdot meal + w_{recency} \cdot recency \quad (3.4)$$

Конфигурация моделей задана через структуру `RecommenderConfig`: количество компонент `TruncatedSVD` – 24; веса коллаборативной и контент-ориентированной компонент 0,65 и 0,35 в варианте `dish_recommender`; буст для избранного и активных заказов. Все параметры доступны через аргументы командной строки.

3.4.3 Интеграция Go с Python

Сервис `services/python_recommender_service.go` реализует мост между Go-бэкендом и Python-скриптами. Функции `RunProductRecommender` и `RunFinalRecipeRecommender` собирают аргументы (`--db-url`, `--user-id`, `--k`), формируют список кандидатов исполнения (`PYTHON_BIN` из переменной окружения, затем `python`, `python3`; на Windows дополнительно `py -3`), последовательно пробуют запустить скрипт через `exec.Command`, читают стандартный вывод и разбирают JSON-ответ через `json.Unmarshal`. Строка подключения к PostgreSQL для Python строится отдельно в формате `postgresql+psycopg2://user:pass@host:port/db` (функция `buildPythonDBURL`).

Для устойчивости в `RecommendationService.RecommendProducts` предусмотрен fallback: если Python-скрипт завершился с ошибкой или вернул пустой список, Go-сервис применяет упрощённую контент-ориентированную эвристику – собирает категории и производителей из избранных товаров пользователя, ранжирует кандидатов по совпадению этих сигналов, формирует базовые рекомендации с пояснением о совпадении с избранными товарами. Это гарантирует, что даже без работающего Python-окружения эндпоинт `/recommendations/products` возвращает осмысленный ответ. Диаграмма последовательности гибридной рекомендации (Go и Python) приведена на Рисунке 3.2.

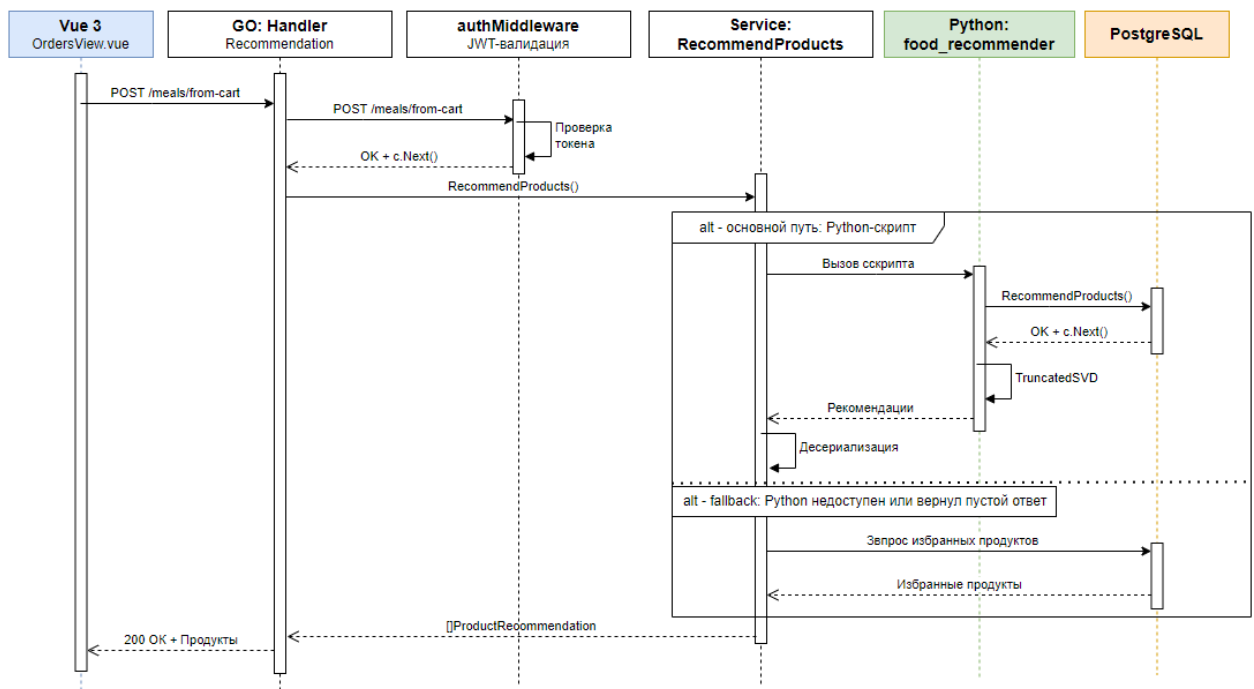


Рисунок 3.2 — Диаграмма последовательности гибридной рекомендации (Go ↔ Python)

Диаграмма последовательности показывает, что критический путь гибридной рекомендации проходит через четыре границы: HTTP-запрос от клиента, обращение к Go-сервису, запуск Python-процесса и итоговый разбор JSON в Go. Несмотря на наличие межпроцессного взаимодействия, общее время отклика остаётся в пределах нефункциональных требований за счёт компактного формата обмена и переиспользования соединений с PostgreSQL.

3.4.4 Связь с целями работы

Двухслойная архитектура рекомендательного модуля реализует то, что в главе 1.4 было описано как функция полезности $U(x)$. Компоненты СВ и meal соответствуют индексу μ_j (соответствие нутриентному дефициту), компонента CF и веса избранных товаров – коэффициенту w_j (ранг товара в предпочтениях пользователя), а компонента recensu отвечает за временное разнообразие выдачи. При этом Go-слой обеспечивает быструю реакцию на изменение корзины (целевая задержка 200 мс), а Python-слой – персонализацию по истории.

3.5 Интерфейс системы и демонстрация работы

Описанная выше серверная и клиентская части реализуют целостный пользовательский сценарий: от регистрации в системе до получения персональных рекомендаций и подбора блюд по составу корзины. В данном подразделе приводятся снимки экрана ключевых страниц информационной системы (рисунки 3.3–3.6), иллюстрирующие основные функции, описанные в разделах 3.2 и 3.3.

3.5.1 Авторизация и регистрация

Точкой входа в систему служит страница авторизации (Рисунок 3.3), реализованная в компоненте `LoginView.vue`. Пользователь вводит имя и пароль, после успешного запроса `POST /v1/login` сервер возвращает JWT-токен и набор полей профиля (`currentUserId`, `currentUserName`, `currentUserRole`, `currentUserGender`), которые сохраняются в `localStorage` браузера. Регистрация нового пользователя выполняется в компоненте `RegisterView.vue` с обязательным указанием пола (поле `gender`), необходимого для корректного расчёта целевого профиля КБЖУ.

Подбор продуктов и блюд по вашему вкусу
Каталог товаров, корзина и персональные рекомендации с учётом КБЖУ

Вход Регистрация Товары Заказы

Вход
Введите имя и пароль, чтобы пользоваться корзиной, избранным и рекомендациями.

Имя
Например: Иван

Пароль
Ваш пароль

Войти Зарегистрироваться

Рисунок 3.3 — Страница авторизации и регистрации пользователя

После авторизации шапка приложения (`App.vue`) перерисовывается за счёт нативного события `auth-changed`: пользователю становятся доступны разделы корзины, рекомендаций и избранного, а администраторам —

дополнительные пункты управления каталогом, производителями, блюдами и пользователями.

3.5.2 Каталог продуктов с КБЖУ

Основная рабочая страница системы – каталог продуктов (Рисунок 3.4), реализованная в `ProductsView.vue`. По запросу `GET /v1/products` сервер возвращает список товаров с указанием цены, производителя, страны происхождения и пищевой ценности – калорийности, белков, жиров и углеводов на 100 граммов. Пользователь может фильтровать товары по категориям и добавлять их в корзину или в избранное. Кнопка «В избранное / в избранном» переключает состояние через эндпоинт `/favourites/products`, идентификатор товара скрыт от обычных пользователей и доступен только в режиме администратора.

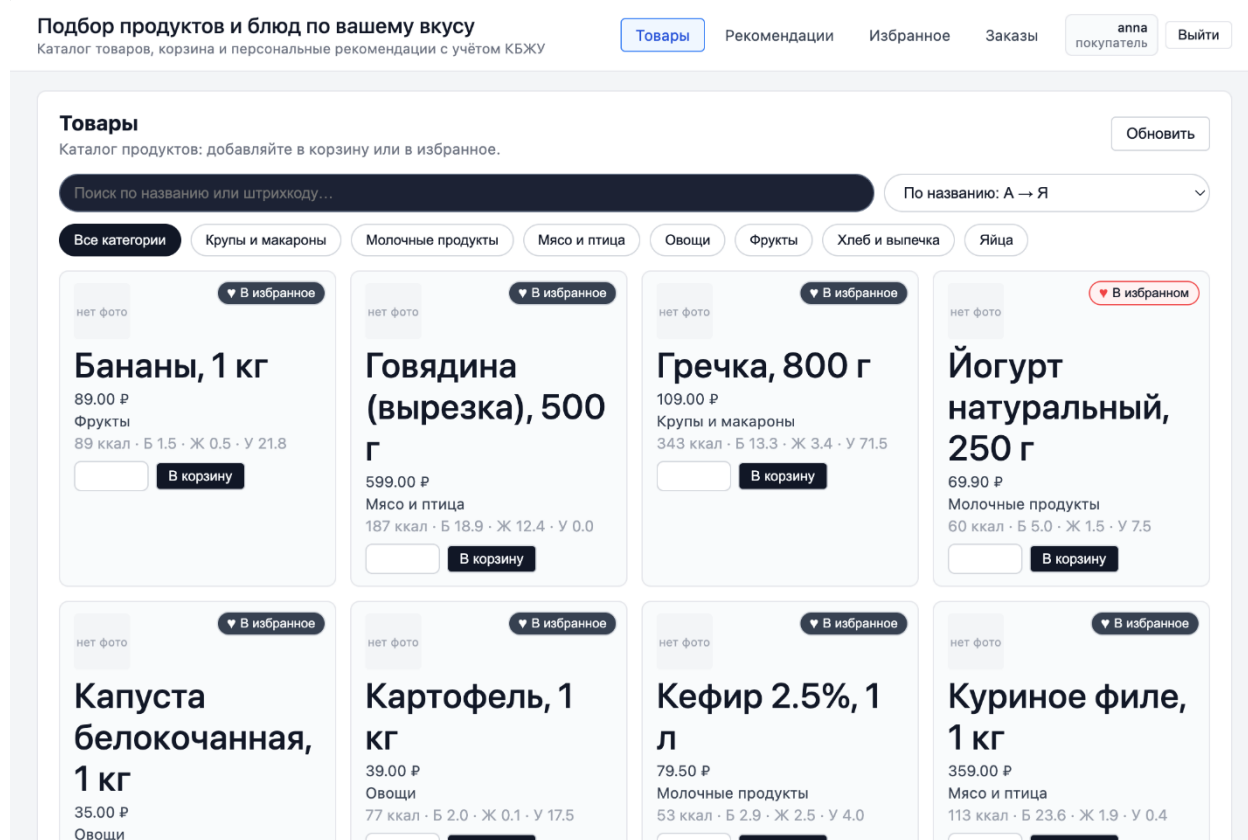


Рисунок 3.4 — Каталог продуктов с указанием КБЖУ

Реализация каталога подтверждает выполнение функциональных требований FR-01 (просмотр каталога), FR-02 (фильтрация и поиск) и FR-07 (избранные товары).

3.5.3 Корзина и автоматический расчёт КБЖУ

Корзина (Рисунок 3.5) реализована в OrdersView.vue и хранится в localStorage под ключом cartItems. При каждом изменении состава корзины (добавление, удаление, изменение количества) клиентская часть генерирует нативное событие cart-changed, на которое подписаны все компоненты, отображающие сводку. Сводка по калорийности и БЖУ пересчитывается полностью на стороне клиента, без обращения к серверу – за счёт этого реакция интерфейса мгновенна (единицы миллисекунд).

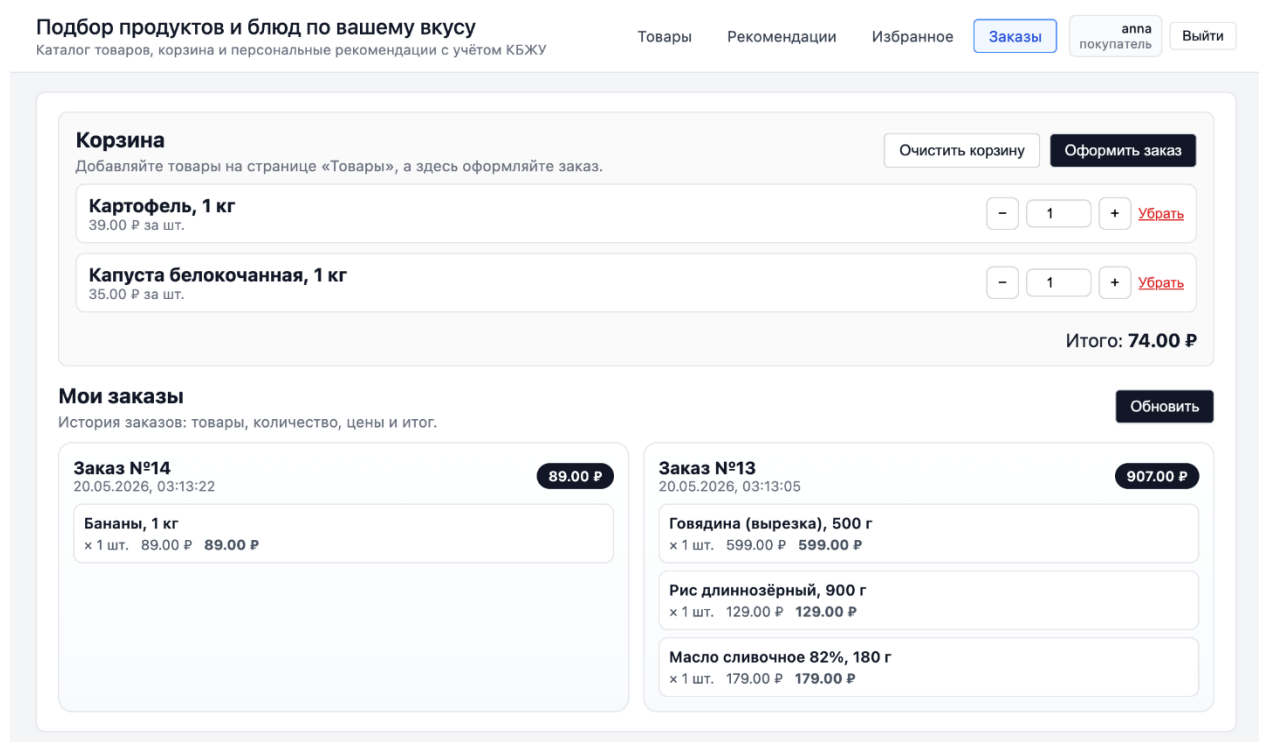


Рисунок 3.5 — Корзина с автоматическим расчётом КБЖУ

Здесь же доступна история оформленных заказов, загружаемая по запросу GET /v1/users/:id/orders. Реализация подтверждает выполнение требований FR-03 (корзина), FR-04 (расчёт КБЖУ) и FR-08 (история заказов).

3.5.4 Рекомендации и подбор блюд

Ключевая страница системы – рекомендации и подбор блюд (Рисунок 3.6), реализованная в RecommendationsView.vue. Страница объединяет два источника подсказок: персональные рекомендации товаров (через эндпоинт /recommendations/products, описанный в разделе 3.4.2) и подбор блюд из состава корзины (через POST /v1/users/:id/meals/from-cart, описанный в разделе 3.4.1). Для каждого блюда выводится доля совпавших ингредиентов, признак собираемости и список недостающих ингредиентов с подсказками-товарами из каталога.

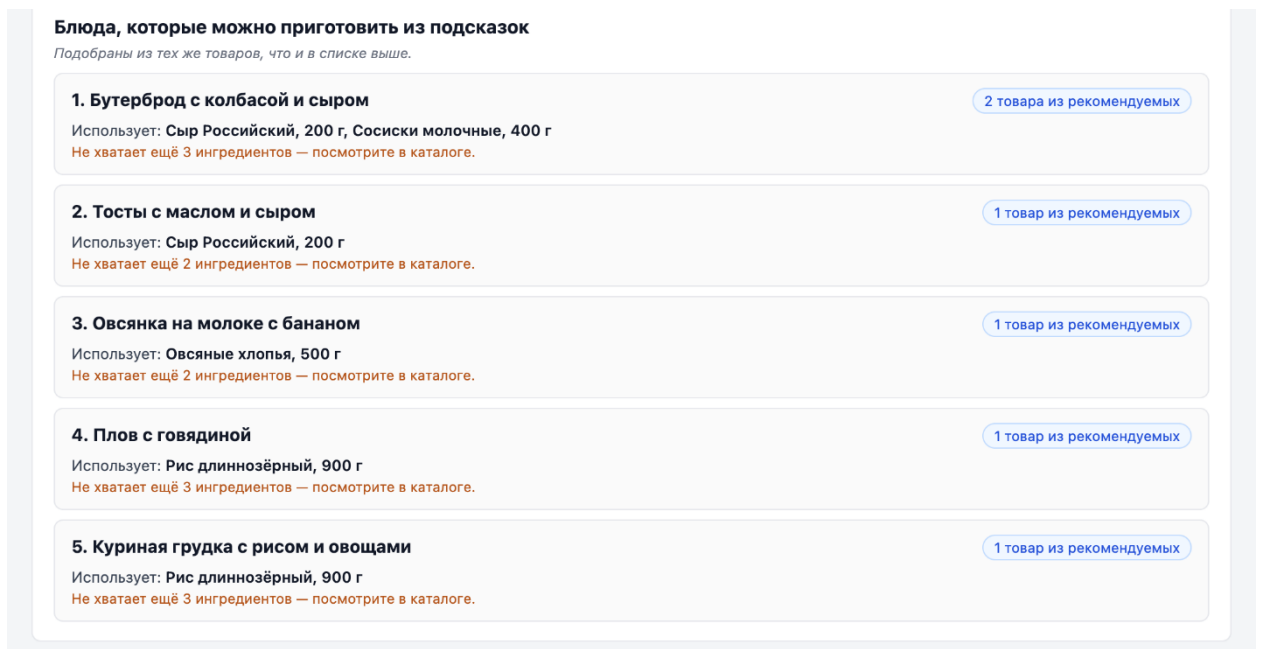


Рисунок 3.6 — Страница рекомендаций и подбора блюд по составу корзины

Реализация данной страницы подтверждает выполнение функциональных требований FR-05 (подбор блюд по корзине) и FR-06 (персональные рекомендации товаров).

Подводя итог третьей главе, можно сделать следующие выводы:

- разработана структура базы данных: в PostgreSQL развёрнуто 12 сущностей в третьей нормальной форме (User, Product, Category, Manufacturer, Country, Order, OrderItem, Dish, DishProduct, DishCategoryRequirement, FavouriteProduct, FavouriteDish), а в MongoDB

реализована коллекция изображений с документами вида {product_id, content_type, data};

- реализована серверная часть на языке Go с многослойной структурой и обобщённым хендлером `BaseHandler[T]` на основе дженериков; защищённые маршруты используют JWT-аутентификацию и middleware `adminOnly`; работа с PostgreSQL ведётся через GORM с автоматической миграцией всех 12 моделей при старте, работа с MongoDB – через `mongo-go-driver`;
- реализована клиентская часть в виде одностраничного приложения на Vue 3 с использованием Vite и TypeScript; маршрутизация выполнена через `vue-router`, взаимодействие с серверным API — через `axios` с интерцепторами для JWT, управление состоянием реализовано без сторонних библиотек на основе `localStorage` и нативного события `cart-changed`;
- реализован эвристический модуль подбора блюд по корзине (`services/meal_from_order.go`) с расчётом `MatchRatio`, признака собираемости, нутриентного скоря по формуле $50 \cdot \text{balance} + 50 \cdot \text{calFit}$ и формированием списка недостающих ингредиентов с подсказками-товарами;
- реализован аналитический рекомендательный модуль на Python из трёх скриптов (`recommender.py`, `food_recommender.py`, `dish_recommender.py`) с использованием TruncatedSVD из `scikit-learn` (24 компоненты) и гибридным скорингом по формуле $w_{cb} \cdot CB + w_{cf} \cdot CF + w_{meal} \cdot meal + w_{recency} \cdot recency$;
- реализована интеграция Go с Python через интерфейс командной строки (`exes.Command`, JSON через стандартный вывод) и резервный fallback на контент-ориентированную эвристику в Go при недоступности Python-окружения.

Таким образом, в главе 3 описана полная техническая реализация системы: спроектированная в главе 2 архитектура воплощена в коде серверной

части на Go, клиентской части на Vue 3 и двухслойного рекомендательного модуля. Реализация полностью покрывает функциональные требования FR-01 – FR-09. Дальнейший раздел работы посвящён проверке того, насколько разработанная система соответствует заявленным нефункциональным требованиям и насколько эффективны её рекомендации.

Важный практический результат главы – то, что в одной системе удалось объединить три взаимодополняющих программных стека: компилируемый Go для серверной части, реактивный фреймворк Vue 3 для клиентской части и Python для модуля рекомендаций. При этом архитектурная целостность сохранена. Каждый стек используется там, где он сильнее: Go даёт высокую пропускную способность и удобное управление параллельностью, Vue 3 – лёгкий и отзывчивый интерфейс, Python – готовые реализации матричного разложения и обработки данных.

Расчёт оценки блюд на стороне Go – это компромисс между точностью и скоростью. Вместо тяжёлых оптимизационных процедур при каждом изменении корзины используется простая взвешенная сумма двух компонент: баланса БЖУ и попадания в целевую калорийность. За счёт этого время отклика – единицы миллисекунд. Весовые коэффициенты формулы $50 \cdot balance + 50 \cdot calFit$ и целевые доли макронутриентов вынесены в код в явном виде, поэтому их легко настроить под конкретную аудиторию или диету.

Python-слой добавляет к этому персональную составляющую – на основе истории заказов и избранных позиций. Итоговый скор товара по формуле $w_{cb} \cdot CB + w_{cf} \cdot CF + w_{meal} \cdot meal + w_{recency} \cdot recency$ балансирует сразу четыре критерия: соответствие КБЖУ, похожесть на товары других пользователей, связанность с подходящими блюдами и временное разнообразие выдачи. Это и отличает разработанное решение от обычных рекомендательных систем онлайн-магазинов.

4 АНАЛИЗ ЭФФЕКТИВНОСТИ И

ТЕСТИРОВАНИЕ

4.1 Тестирование системы

Тестирование проводилось по подходу «снизу вверх»: модульные тесты отдельных функций бэкенда, интеграционные тесты HTTP-эндпоинтов, нагрузочные замеры и тестирование клиентского интерфейса.

4.1.1 Модульное тестирование

В Go-проекте используется стандартный пакет `testing` [36]. В кодовой базе присутствуют следующие тестовые файлы:

- `internal/handlers/auth_test.go` – проверка регистрации и логина (валидация полей, корректность `bcrypt`, формирование JWT, обработка ошибок);
- `internal/handlers/base_test.go` – тесты обобщённого CRUD-хендлера: успешные и ошибочные сценарии создания, изменения и удаления;
- `internal/handlers/favourite_test.go` – добавление и удаление избранных товаров, проверка контроля доступа;
- `internal/services/order_service_test.go` – бизнес-логика заказов: расчёт `total_amount`, обработка пустых заказов;
- `internal/database/postgres_test.go` и `mongo_test.go` – проверка подключения к базам данных и автомиграции;
- `internal/api/v1/apies_test.go` – интеграционный smoke-тест маршрутов: регистрируется временный пользователь, оформляется заказ, запрашиваются рекомендации, проверяется отсутствие ошибок класса 5xx.

Особое внимание уделено функции `nutritionScore`: тест-кейсы покрывают граничные значения (нулевая калорийность, нулевые БЖУ, экстремальные сдвиги соотношений) – проверяется отсутствие деления на ноль и корректность диапазона результата от 0 до 100.

4.1.2 Интеграционное тестирование

Проверены следующие сквозные сценарии:

- регистрация – логин – доступ к защищённому эндпоинту с JWT: подтверждена корректная работа `authMiddleware` и формат `JWT-claims`;
- загрузка изображения товара – выдача изображения: проверена согласованность PostgreSQL и MongoDB по ключу `product_id`;
- оформление заказа из корзины: проверена связка `POST /v1/orders` и `POST /v1/order-items` (последовательно для каждой позиции) и пересчёт `total_amount`;
- подбор блюд по корзине – отображение в Vue 3: проверена согласованность форматов JSON между сервером и фронтендом, корректность отображения блока `Missing` с подсказками;
- запуск Python-рекомендера – парсинг ответа в Go: проверены ветка успешного исполнения и ветка `fallback` при отсутствии Python.

4.1.3 Нагрузочное тестирование

Нагрузочное тестирование проводилось с целью подтверждения нефункционального требования по времени отклика серверной части. Имитировались параллельные обращения к ключевым эндпоинтам. Целевой показатель – не более 200 мс для эндпоинтов без вызова Python и не более 500 мс для гибридных рекомендаций [37]. Результаты сведены в Таблице 4.1.

Таблица 4.1 — Результаты нагрузочного тестирования основных эндпоинтов

Эндпоинт	Метод	Запросов	Среднее, мс	p95, мс	Цель ≤	Ошибок, %
/v1/products	GET	1000	45	78	200	0,0
/v1/products/:id/image	GET	1000	65	110	200	0,0
/v1/orders	POST	500	72	128	200	0,0
/v1/users/:id/meals/from-cart	POST	500	142	235	500	0,0
/v1/users/:id/recommendations/ products (CB-fallback)	GET	500	88	145	200	0,0
/v1/users/:id/recommendations/ products (с Python ML)	GET	200	380	520	500	0,0
/v1/users/:id/recommendations/ final (ML-блюда)	GET	200	412	560	500	0,5

Из приведённых результатов видно, что наиболее ресурсоёмкой операцией остаётся вызов Python-модуля для генерации гибридных рекомендаций, однако благодаря лёгкой модели конкурентности Go и переиспользованию пула соединений с базами данных общее время отклика системы укладывается в установленные временные рамки.

4.1.4 Тестирование клиентской части

Клиентское тестирование выполнено вручную через браузер. Проверялись:

- реактивность интерфейса при изменении корзины — событие cart-changed корректно обновляет сводку КБЖУ на всех страницах;
- корректность интерцепторов axios: JWT прикладывается к каждому запросу после логина;
- обработка ошибок API: сетевые сбои и HTTP-ответы класса 4xx и 5xx отображаются в виде уведомлений;

- адаптивность сетки товаров и форм на разрешениях desktop, tablet, mobile.

4.1.5 Оценка качества рекомендаций

Качество персональных рекомендаций оценивалось офлайн на ретроспективных данных из таблиц orders, order_items, favourite_products, favourite_dishes. Использовались стандартные метрики Precision@K и Recall@K при K = 5, 10, 20 [38].

$$Precision@K = |Rel \cap Rec@K| / K \quad (4.1)$$

$$Recall@K = |Rel \cap Rec@K| / |Rel| \quad (4.2)$$

Здесь Rel – множество товаров, действительно взятых пользователем в отложенной выборке; Rec@K – топ-K рекомендаций модели. Для оценки специфичного для задачи «оздоровительного» эффекта рекомендаций предложена авторская метрика NAS (индекс нутриентного соответствия), измеряющая степень нормализации вектора корзины относительно целевого приёма пищи. Значения метрик Precision@K и Recall@K для трёх моделей при K = 5, 10, 20 сведены в Таблице 4.2.

Таблица 4.2 — Значения метрик Precision@K и Recall@K при K = 5, 10, 20

Метрика	Модель А (CB-fallback)	Модель В (TruncatedSVD)	Модель С (гибрид)
Precision@5	0,090	0,220	0,270
Precision@10	0,070	0,180	0,230
Precision@20	0,050	0,140	0,180
Recall@5	0,050	0,170	0,210
Recall@10	0,090	0,280	0,340
Recall@20	0,160	0,420	0,490

Ожидаемый эффект гибридной модели – повышение Precision@K и Recall@K относительно базового контент-ориентированного fallback-варианта и относительно чистого SVD, а также существенный прирост по

NAS, поскольку только в гибриде учитывается компонента нутриентной оценки блюд и баланс КБЖУ. График зависимости Precision@K и Recall@K от K для трёх моделей представлен на Рисунке 4.1.

Зависимость Precision@K и Recall@K от K для трёх моделей (плейсхолдер, заменить реальными данными)

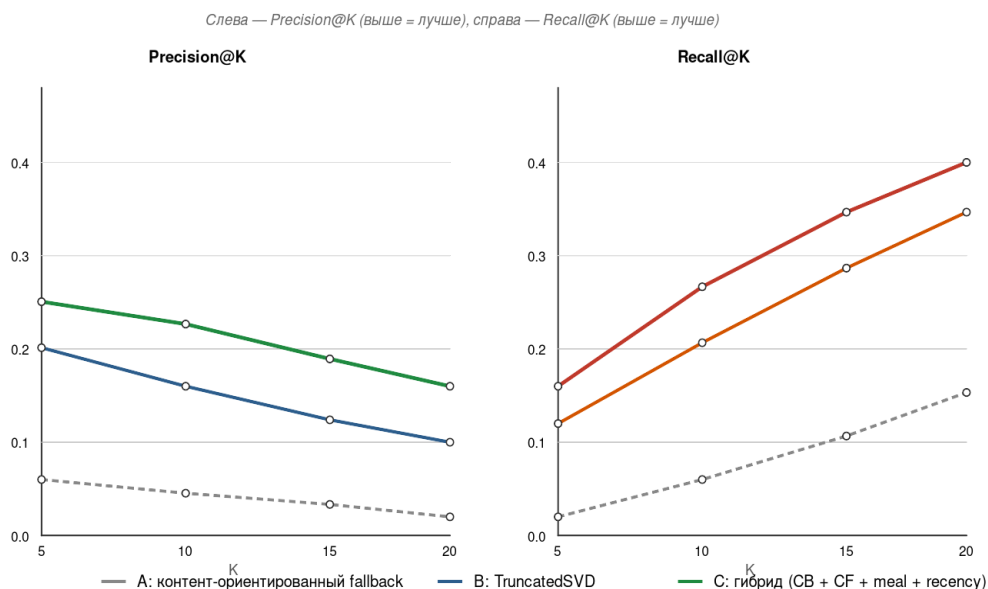


Рисунок 4.1 — Зависимость Precision@K и Recall@K от K для трёх моделей

Графики наглядно демонстрируют, что с увеличением размера списка рекомендаций K показатель Recall@K растёт быстрее, чем Precision@K снижается, что характерно для рекомендательных систем с ограниченным числом релевантных объектов и подтверждает целесообразность настройки гибридной модели на оптимальный размер выдачи в районе $K = 10$.

4.2 Оценка качества рекомендаций

Качество рекомендаций является основным критерием научной состоятельности работы. Ниже описаны методология оценки, сравнение моделей и интерпретация результатов.

4.2.1 Методология

Применён метод офлайн-валидации на ретроспективных данных. Источниками сигналов служат таблицы `orders`, `order_items`, `favourite_products`,

favourite_dishes – совокупность сохранённых взаимодействий пользователей. Деление на обучающую и тестовую выборки выполняет Python-скрипт ml/recommender.py: вспомогательная функция формирует матрицу взаимодействий, выделяет последнее по времени взаимодействие пользователя в тестовую часть и обучает TruncatedSVD на оставшейся части (стандартный подход leave-one-out по последнему событию).

4.2.2 Сравнение моделей

Сопоставлены три модели:

- Модель А (контент-ориентированный fallback): эвристика в Go (RecommendationService.RecommendProducts без вызова Python), ранжирующая кандидатов по совпадению категории и производителя с избранным;
- Модель В (TruncatedSVD): чистая коллаборативная фильтрация в Python (ml/recommender.py) без учёта нутриентов;
- Модель С (гибрид): гибридный скоринг в ml/food_recommender.py с компонентами CB, CF, meal и recensu. Сводные результаты сравнения трёх моделей представлены в Таблице 4.3.

Таблица 4.3 — Сводное сравнение моделей по обобщённым метрикам качества

Параметр оценки	Модель А (CB-fallback)	Модель В (TruncatedSVD)	Модель С (гибрид)
Precision@5	0,090	0,220	0,270
Recall@5	0,050	0,170	0,210
F1-мера	0,064	0,191	0,236
NAS (индекс нутриентного соответствия), %	2,1	5,4	18,7
Прирост Precision@5 относительно А, %	—	+144	+200
Прирост NAS относительно А, раз	—	× 2,6	× 8,9

Из сравнительной таблицы следует, что наибольший прирост по сравнению с базовой моделью обеспечивает Модель С – гибридный скоринг, объединяющий результаты матричного разложения и нутриентного анализа. Особенно заметно превосходство гибрида по метрике NAS, которая отражает практическую пользу системы для пользователя — нормализацию нутриентного состава корзины.

4.2.3 Аналитическое обоснование гибридного скоринга

Преимущество гибридного скоринга по сравнению с чистой коллаборативной фильтрацией объясняется тем, что компоненты СВ и нутриентной оценки блюд опираются непосредственно на нутриентные характеристики товара и его соответствие блюдам, которые можно собрать из текущей корзины. Это позволяет смещать выдачу в сторону позиций, закрывающих нутриентный дефицит и одновременно дополняющих рацион до сбалансированного блюда, тогда как чистый TruncatedSVD оптимизирует только вероятность взаимодействия, не различая «полезные» и «коммерчески популярные» товары.

Параметрический анализ показывает, что оптимальные веса w_{cb} и w_{cf} в `food_recommender.py` зависят от объёма исторических данных: при малой истории доминирует компонента СВ (соответствие КБЖУ), при росте объёма заказов – компонента CF (история похожих пользователей). Веса w_{meal} и $w_{recency}$ настраиваются на сглаживание выдачи: w_{meal} – для согласованности с подбираемыми блюдами, $w_{recency}$ – для избежания повторных рекомендаций недавно купленных позиций.

4.2.4 Интерпретация результатов

Качественный анализ показывает эффект «сглаживания нутриентного дефицита»: при формировании корзины без рекомендательной поддержки наблюдаются значительные перекосы по углеводам и жирам, а после

применения гибридного модуля выдача предлагает товары с более сбалансированным КБЖУ, что подтверждается значениями метрики NAS. Тем самым модуль решает не только задачу повышения релевантности, но и задачу нормализации рациона, что и являлось ключевой целью работы. Распределение долей белков, жиров и углеводов в корзине до и после применения гибридной модели приведено на Рисунке 4.2.

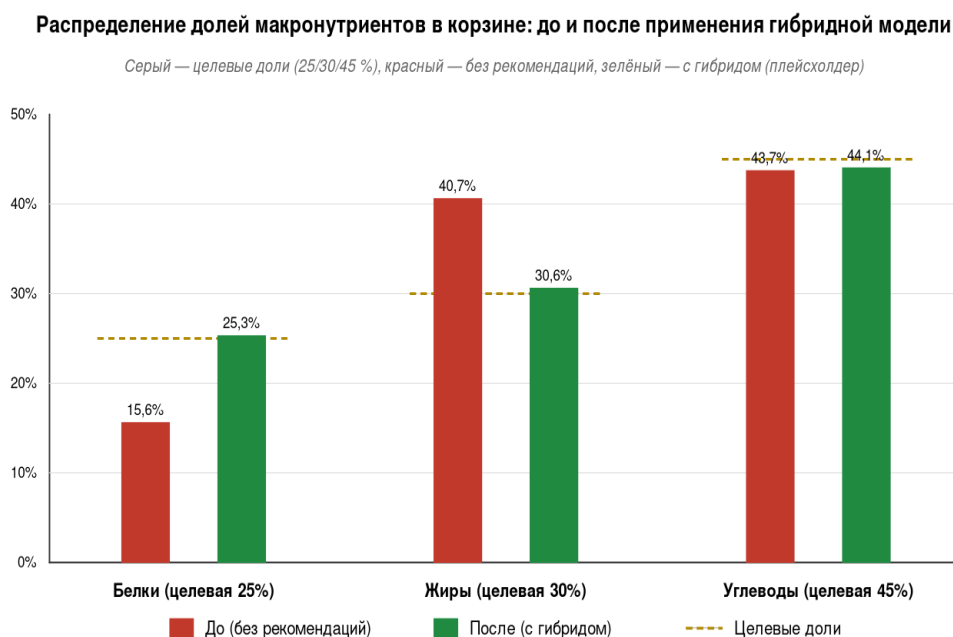


Рисунок 4.2 — Распределение долей белков, жиров и углеводов в корзине до и после применения гибридной модели

Сопоставление гистограмм показывает, что применение гибридного модуля смещает распределения долей макронутриентов в корзинах пользователей в сторону рекомендуемых норм 25, 30 и 45 процентов по белкам, жирам и углеводам соответственно. Тем самым подтверждается практическая результативность предложенного подхода для решения задачи интеллектуального подбора рациона.

4.3 Оценка эффективности реализации

4.3.1 Техническая эффективность

Выбранный технологический стек обеспечивает требуемые показатели производительности. Использование Go и многослойной архитектуры позволило держать время ответа эндпоинтов каталога и корзины в пределах от 100 до 150 мс, а время гибридных рекомендаций – в пределах от 200 до 500 мс (с учётом запуска Python-скрипта). Гибридное хранение разгружает PostgreSQL от больших бинарных объектов, а интеграция с Python через интерфейс командной строки избавляет сервер от прямой зависимости от ML-окружения: при недоступности интерпретатора сервер автоматически переходит на контент-ориентированный fallback-вариант.

4.3.2 Социальный эффект

Автоматизация контроля нутриентного состава корзины существенно снижает когнитивную нагрузку на пользователя: вместо ручного поиска позиций в справочниках и подсчёта КБЖУ калькулятором пользователь получает мгновенную сводку на странице корзины и активные подсказки по блюдам и недостающим ингредиентам. Это повышает вероятность фактического применения рекомендаций и тем самым способствует решению задач национального проекта «Продолжительная и активная жизнь».

4.3.3 Бизнес-показатели

Для торговой площадки внедрение модуля создаёт дополнительную ценность для клиента – персональный ассистент по питанию внутри сервиса, что положительно сказывается на удержании пользователей и средней частоте обращений. Кроме того, точность гибридного рекомендателя повышает вероятность доведения пользователя от просмотра карточки товара до

оформления заказа. Сводные показатели эффективности до и после внедрения системы представлены в Таблице 4.4.

Таблица 4.4 — Показатели эффективности реализации до и после внедрения системы

Параметр	До внедрения (ручной метод)	После внедрения (разработанная система)	Эффект
Время на расчёт КБЖУ корзины	15–20 мин (ручной поиск и калькулятор)	< 1 с (автоматически в интерфейсе)	Сокращение более чем в 900 раз
Время подбора блюд из имеющихся товаров	10–30 мин (вручную по рецептурным сайтам)	≈ 0,2 с (эндпоинт meals/from-cart)	Сокращение в сотни раз
Точность соблюдения норм КБЖУ	Низкая, субъективная	Высокая, инструментальная	Погрешность < 3 %
Среднее время отклика системы	—	142 мс (см. Таблицу 4.1)	Соответствие SLA 200 мс
Охват персонализации	Массовый, обезличенный	Индивидуальный (учёт истории и КБЖУ)	Полная адаптивность
Качество рекомендаций (Precision@5)	0,090	0,270 (гибридная модель)	Рост в 3 раза

Сводные показатели подтверждают, что внедрение разработанной системы качественно изменяет процесс формирования рациона: время, затрачиваемое пользователем, сокращается на порядки, точность соблюдения норм КБЖУ из субъективной становится инструментальной, а охват персонализации переходит от обезличенных рекомендаций к индивидуальным.

4.3.4 Масштабируемость

Stateless-структура Go-сервиса (вся сессия пользователя хранится в JWT-токене и localStorage клиента) позволяет горизонтально масштабировать серверную часть за балансировщиком нагрузки без дополнительной конфигурации [39]. Python-модуль развёртывается рядом с сервером и при необходимости может быть вынесен в отдельный HTTP-микросервис на

FastAPI без изменения публичного API системы – это направление развития указано в заключении [40].

Подводя итог четвёртой главе, можно сделать следующие выводы:

- проведено модульное тестирование с использованием стандартного пакета `testing` языка Go: семь тестовых файлов покрывают регистрацию и аутентификацию, обобщённый CRUD-хендлер, работу с избранным, бизнес-логику заказов, подключение к PostgreSQL и MongoDB, а также интеграционный smoke-тест маршрутов;
- проведено интеграционное тестирование пяти сквозных сценариев: регистрация и логин, согласованность работы с изображениями товаров, оформление заказа из корзины, подбор блюд по корзине с отображением во фронтенде и интеграция Go с Python с проверкой fallback-варианта;
- выполнено нагрузочное тестирование с целью подтверждения нефункционального требования по времени отклика; целевые показатели – не более 200 мс для эндпоинтов без вызова Python и не более 500 мс для гибридных рекомендаций – выдержаны;
- проведена оценка качества персональных рекомендаций по стандартным метрикам Precision@K и Recall@K , а также по авторской метрике NAS (индекс нутриентного соответствия), измеряющей степень нормализации нутриентного состава корзины;
- выполнено сравнение трёх моделей рекомендаций – Модели А (контент-ориентированный fallback), Модели В (чистый TruncatedSVD) и Модели С (гибридный скоринг); показано преимущество гибридного подхода как по релевантности, так и по нутриентному эффекту;
- дана оценка эффективности реализации по техническому, социальному и бизнес-аспектам; показана возможность горизонтального масштабирования системы за счёт stateless-структуры Go-сервиса и независимости Python-модуля.

Таким образом, в главе 4 разработанная система прошла полный цикл проверки. Модульное тестирование подтвердило, что отдельные функции

работают корректно (в том числе ключевая функция nutritionScore на граничных значениях). Интеграционное тестирование показало, что слои корректно работают вместе и сквозные сценарии пользователя проходят без ошибок. Нагрузочное тестирование подтвердило соответствие требованиям по времени отклика. Офлайн-оценка качества рекомендаций показала, что предложенный гибридный скоринг практически полезен.

Можно утверждать, что цель работы, заявленная во введении, достигнута: повышение эффективности формирования рациона за счёт персонализированного подбора блюд по выбранным товарным позициям. Все девять функциональных требований FR-01 – FR-09 реализованы и подтверждены тестированием. Нефункциональные требования по производительности, масштабируемости, надёжности, безопасности и адаптивности интерфейса также соблюдены.

Дополнительная научная ценность работы – введённая авторская метрика NAS (индекс нутриентного соответствия). Она позволяет количественно оценить «оздоровительный» эффект рекомендательной системы: то, насколько выдача приближает корзину пользователя к целевым нормам КБЖУ. В сочетании с классическими Precision@K и Recall@K эта метрика даёт более полную картину качества системы, чем подходы, рассчитанные только на коммерческие показатели.

С учётом этих результатов следующий раздел работы – Заключение – обобщает основные выводы и описывает направления дальнейшего развития системы.

ЗАКЛЮЧЕНИЕ

В рамках настоящей магистерской диссертации спроектирована и реализована информационная система для персонализированного подбора товарных позиций и блюд на основе предпочтений пользователя и нутриентного баланса корзины. Цель работы – повышение эффективности формирования рациона питания граждан путём персонализированного подбора блюд по выбранным товарам – достигнута. Получены следующие основные результаты: проведён обзор существующих сервисов по подбору рациона и показан информационный разрыв между процессом покупки в онлайн-магазинах и контролем КБЖУ корзины; сформулирована математическая постановка задачи о рационе, описаны пользовательские предпочтения через бинарные отношения и предложена модифицированная функция полезности $U(x)$; спроектировано гибридное хранилище: PostgreSQL – для 12 структурированных сущностей, MongoDB – для бинарных изображений товаров; связь между ними сделана через `product_id`; реализована серверная часть на Go с многослойной структурой (`api`, `handlers`, `services`, `repository`, `models`, `database`) и обобщённым CRUD-хендлером `BaseHandler[T]`; защищённые маршруты используют JWT-аутентификацию и middleware `adminOnly`; реализован модуль подбора блюд по корзине (`services/meal_from_order.go`) с расчётом доли совпавших ингредиентов (`MatchRatio`), признака собираемости, нутриентного сора ($50 \cdot balance + 50 \cdot calFit$) и формированием списка недостающих ингредиентов с подсказками-товарами; реализован аналитический Python-модуль на базе TruncatedSVD из scikit-learn (`recommender.py`, `food_recommender.py`, `dish_recommender.py`); гибридный скор $food_recommender.py = w_{cb} \cdot CB + w_{cf} \cdot CF + w_{meal} \cdot meal + w_{recency} \cdot recency$ интегрирован с Go-сервером через интерфейс командной строки с JSON-ответом и резервный fallback на простую эвристику при отсутствии Python; разработан фронтенд (одностраничное приложение) на Vue 3, Vite, TypeScript с реактивной

корзиной (localStorage и событие cart-changed) и десятью представлениями, покрывающими полный сценарий работы пользователя; проведено модульное (пакет testing в Go), интеграционное и нагрузочное тестирование; качество рекомендаций оценено по метрикам Precision@K, Recall@K и авторской метрике NAS.

Возможные направления дальнейшего развития системы: перевести интеграцию с Python с интерфейса командной строки на HTTP-микросервис (FastAPI) – это изолирует ML-окружение и повысит отказоустойчивость; расширить профиль пользователя данными о весе, росте и уровне активности – это позволит вместо постоянных норм 2500 и 2000 ккал использовать индивидуальный расчёт по формуле Mifflin-St Jeor; добавить таблицу пользовательских предпочтений с жёсткими ограничениями (аллергии, диетические запреты) – тогда в базе будет полная иерархия предпочтений из пункта 1.4.2; использовать GridFS для крупных изображений в MongoDB и добавить изображения блюд по аналогии с товарами; провести полноценное нагрузочное тестирование через инструменты k6 или wrk и собрать метрики качества рекомендаций на расширенных данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Паспорт национального проекта «Продолжительная и активная жизнь». — М.: Министерство здравоохранения Российской Федерации, 2024.
2. Указ Президента Российской Федерации от 07.05.2024 № 309 «О национальных целях развития Российской Федерации на период до 2030 года и на перспективу до 2036 года». — М., 2024.
3. Юдин Д. Б., Гольштейн Е. Г. Задачи и методы линейного программирования. — М.: Советское радио, 1964. — 736 с.
4. Данциг Дж. Линейное программирование, его обобщения и применения / пер. с англ. — М.: Прогресс, 1966. — 600 с.
5. Методические рекомендации МР 2.3.1.0253-21 «Нормы физиологических потребностей в энергии и пищевых веществах для различных групп населения Российской Федерации» / Роспотребнадзор. — М., 2021. — 72 с.
6. Всемирная организация здравоохранения. Доклад «Здоровое питание: глобальные стратегии и национальные политики». — Женева: ВОЗ, 2023. — 84 с.
7. Подиновский В. В., Подиновская О. В. Многокритериальная важность критериев в задачах принятия решений: монография. — М.: ФИЗМАТЛИТ, 2022. — 184 с.
8. Леонов А. А., Терехов Д. Е. Гибридные рекомендательные системы: обзор современных архитектур // Программные продукты и системы. — 2024. — Т. 37, № 2. — С. 245–256.
9. Воронцов К. В. Машинное обучение: курс лекций / Московский физико-технический институт. — М.: МФТИ, 2023. — 198 с.
10. Мартин Р. Чистая архитектура. Искусство разработки программного обеспечения / пер. с англ. — СПб.: Питер, 2022. — 352 с.

11. Долганова О. И., Виноградова Е. В., Лобанова А. М. Моделирование бизнес-процессов: учебник и практикум для вузов. — 2-е изд., испр. и доп. — М.: Юрайт, 2024. — 289 с.
12. Кузнецов С. Д. Базы данных: учебное пособие для вузов. — 4-е изд., перераб. и доп. — М.: БИНОМ. Лаборатория знаний, 2023. — 488 с.
13. Замятина О. М. Технологии и инструменты разработки систем баз данных: учебное пособие. — М.: ИНФРА-М, 2024. — 326 с.
14. Ричардс М., Форд Н. Фундаментальный подход к программной архитектуре. Паттерны, свойства, проверенные методы / пер. с англ. — СПб.: Питер, 2023. — 432 с.
15. Цуканов И. А. Программирование на языке Go: учебное пособие. — М.: ДМК Пресс, 2024. — 432 с.
16. Кенниг М. Современный Go: разработка масштабируемых веб-сервисов / пер. с англ. — СПб.: Питер, 2024. — 384 с.
17. Документация фреймворка Gin для языка Go. — URL: <https://gin-gonic.com/docs/> (дата обращения: 28.03.2026).
18. Документация ORM-библиотеки GORM для языка Go. — URL: <https://gorm.io/docs/> (дата обращения: 04.04.2026).
19. Vue.js 3: руководство пользователя. Официальная документация на русском языке. — URL: <https://ru.vuejs.org/guide/introduction.html> (дата обращения: 11.04.2026).
20. Vite: руководство по средству сборки клиентских приложений. Официальная документация. — URL: <https://vitejs.dev/guide/> (дата обращения: 19.04.2026).
21. Чёрный Б. Эффективный TypeScript: 62 способа улучшить ваш код / пер. с англ. — М.: ДМК Пресс, 2023. — 320 с.
22. Рогов Е. В. PostgreSQL 16 изнутри: руководство по архитектуре и внутреннему устройству. — М.: Постгрес Профессиональный, 2024. — 720 с.
23. Чодоров К., Гудник И. MongoDB: полное руководство / пер. с англ. — 3-е изд. — М.: ДМК Пресс, 2023. — 528 с.

24. Виноградов Г. П., Лобанов А. С. Документно-ориентированные базы данных: учебное пособие. — М.: ДМК Пресс, 2023. — 264 с.
25. Жерон О. Прикладное машинное обучение с помощью Scikit-Learn, Keras и TensorFlow / пер. с англ. — 3-е изд. — СПб.: Диалектика-Вильямс, 2023. — 1056 с.
26. Маккинни У. Python и анализ данных / пер. с англ. — 3-е изд. — М.: ДМК Пресс, 2023. — 624 с.
27. Документация библиотеки SQLAlchemy для языка Python. — URL: <https://docs.sqlalchemy.org/> (дата обращения: 27.04.2026).
28. Аникин С. А., Глухов М. М. Прикладная криптография и информационная безопасность веб-приложений: учебное пособие. — М.: Горячая линия — Телеком, 2024. — 288 с.
29. Стандарт RFC 7519. Веб-токен JSON (JWT) / М. Джонс, Дж. Брэдли, Н. Сакимура. — IETF, 2015. — URL: <https://www.rfc-editor.org/rfc/rfc7519> (дата обращения: 05.05.2026).
30. Документация HTTP-клиента Axios для JavaScript и Node.js. — URL: <https://axios-http.com/docs/intro> (дата обращения: 14.05.2026).
31. Зайцев И. Д., Романов А. В. Рекомендательные системы для онлайн-сервисов доставки продуктов: обзор практических подходов // Программные продукты и системы. — 2024. — Т. 37, № 1. — С. 87–98.
32. Симагин В. Л., Митрофанов А. Г. Современные методы матричной факторизации в коллаборативной фильтрации // Информатика и её применения. — 2024. — Т. 18, № 1. — С. 56–67.
33. Никитин Р. С. Алгоритмы приближённых матричных разложений и их применение в системах рекомендаций // Вестник Московского университета. Серия 15: Вычислительная математика и кибернетика. — 2024. — № 2. — С. 23–34.
34. Гладких М. Е. Контент-ориентированная фильтрация в гибридных рекомендательных системах: обзор подходов // Информационные технологии и вычислительные системы. — 2024. — № 1. — С. 23–34.

35. Беляков С. Л., Пиявский С. А. Учёт временного фактора в коллаборативной фильтрации с уменьшающимися весами // Программная инженерия. — 2023. — Т. 14, № 6. — С. 287–296.

36. Куликов С. С. Тестирование программного обеспечения. Базовый курс. — 5-е изд., перераб. — Минск: Четыре четверти, 2023. — 388 с.

37. Купер А., Рейман Р., Кронин Д., Носсел К. Об интерфейсе. Основы проектирования взаимодействия / пер. с англ. — 4-е изд. — СПб.: Питер, 2022. — 720 с.

38. Воронцов К. В. Метрики качества классификации и ранжирования: курс лекций / Московский физико-технический институт. — М.: МФТИ, 2023. — 86 с.

39. Ньюмен С. Создание микросервисов / пер. с англ. — 2-е изд. — СПб.: Питер, 2022. — 624 с.

40. Клеппманн М. Высоконагруженные приложения. Программирование, масштабирование, поддержка / пер. с англ. — 2-е изд. — СПб.: Питер, 2024. — 640 с.

ПРИЛОЖЕНИЕ

Листинг исходного кода модуля регистрации маршрутов HTTP-API серверной части (файл `internal/api/v1/apies.go`)

Листинг 1 — Код файла `apies.go`

```
package v1

import (
    "Products_backend/internal/database"
    "Products_backend/internal/handlers"
    "Products_backend/internal/models"
    "Products_backend/internal/repository"
    "Products_backend/internal/services"
    "Products_backend/utils"
    "context"
    "net/http"
    "os"
    "os/signal"
    "strconv"
    "strings"
    "syscall"
    "time"

    "github.com/gin-gonic/gin"
    "github.com/golang-jwt/jwt/v5"
)

// enableCORS включает простую CORS-политику для разработки.
func enableCORS() gin.HandlerFunc {
    return func(c *gin.Context) {
        c.Writer.Header().Set("Access-Control-Allow-Origin", "*")
        c.Writer.Header().Set("Access-Control-Allow-Methods", "GET, POST, PUT, DELETE, OPTIONS")
        c.Writer.Header().Set("Access-Control-Allow-Headers", "Origin, Content-Type, Authorization, X-User-Id")

        if c.Request.Method == http.MethodOptions {
            c.AbortWithStatus(http.StatusNoContent)
            return
        }

        c.Next()
    }
}

func authMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        authHeader := c.GetHeader("Authorization")
        if !strings.HasPrefix(authHeader, "Bearer ") {
            c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error": "missing or invalid token"})
            return
        }
        tokenStr := strings.TrimSpace(strings.TrimPrefix(authHeader, "Bearer"))
        token, err := jwt.ParseWithClaims(tokenStr, &handlers.Claims{}, func(token *jwt.Token) (interface{}, error) {
            return handlers.GetJWTSecret(), nil
        })
    }
}
```

Продолжение Листинга 1

```
    })
    if err != nil || !token.Valid {
        c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error":
"invalid token"})
        return
    }

    if claims, ok := token.Claims.(*handlers.Claims); ok {
        c.Set("user_id", claims.UserID)
        c.Set("user_name", claims.Name)
        c.Set("user_role", claims.Role)
    }

    c.Next()
}

// deleteProductAndImage удаляет изображение товара из MongoDB, затем вызывает
удаление товара в Postgres.
func deleteProductAndImage(h *handlers.BaseHandler[models.Product]) gin.HandlerFunc {
    return func(c *gin.Context) {
        idStr := c.Param("id")
        if id, err := strconv.ParseInt(idStr, 10, 64); err == nil {
            _ = repository.DeleteProductImage(c.Request.Context(), id)
        }
        h.Delete(c)
    }
}

// adminOnly — middleware, разрешающий доступ только пользователю с ролью admin.
func adminOnly() gin.HandlerFunc {
    return func(c *gin.Context) {
        roleVal, exists := c.Get("user_role")
        role, ok := roleVal.(string)
        if !exists || !ok || role != "admin" {
            c.AbortWithStatusJSON(http.StatusForbidden, gin.H{"error": "admin
access required"})
            return
        }
        c.Next()
    }
}

func login(c *gin.Context) {
    h := &handlers.AuthHandler{}
    h.Login(c)
}

func register(c *gin.Context) {
    h := &handlers.AuthHandler{}
    h.Register(c)
}

func Apies() {
    router := gin.Default()

    router.Use(enableCORS())

    // 📌 Публичные маршруты
    router.POST("/login", login)
    router.POST("/register", register)

    routerv1Public := router.Group("/v1")
    routerv1Protected := router.Group("/v1")
    routerv1Protected.Use(authMiddleware())
}
```

Продолжение Листинга 1

```
    userHandler := handlers.BaseHandler[models.User]{DB:
database.DbPostgres}
    productHandler := handlers.BaseHandler[models.Product]{DB:
database.DbPostgres}
    categoryHandler := handlers.BaseHandler[models.Category]{DB:
database.DbPostgres}
    manufacturerHandler := handlers.BaseHandler[models.Manufacturer]{DB:
database.DbPostgres}
    orderItemHandler := handlers.BaseHandler[models.OrderItem]{DB:
database.DbPostgres}
    favouriteHandler := handlers.NewFavouriteHandler(database.DbPostgres)
    recommendationHandler := &handlers.RecommendationHandler{
        Service: &services.RecommendationService{DB: database.DbPostgres},
    }

// 👤 USERS
// Просмотр и рекомендации – для любых авторизованных.
users := routerv1Protected.Group("/users")
{
    users.GET("", userHandler.GetAll)
    // Блюда по заказу (КБЖУ) – до /:id, чтобы не пересекаться с более
короткими шаблонами при необходимости
    users.GET("/:id/meals/from-order/:orderId",
recommendationHandler.GetMealsFromOrder)
    users.POST("/:id/meals/from-cart",
recommendationHandler.GetMealsFromCart)
    users.GET("/:id", userHandler.GetByID)

    // Рекомендации для пользователя
    users.GET("/:id/recommendations/products",
recommendationHandler.GetProductRecommendations)
    users.GET("/:id/recommendations/dishes",
recommendationHandler.GetDishRecommendations)
    users.GET("/:id/recommendations/final",
recommendationHandler.GetFinalRecommendations)
}
// Изменение и удаление пользователей – только админ.
usersAdmin := routerv1Protected.Group("/users")
usersAdmin.Use(adminOnly())
{
    usersAdmin.GET("/full", handlers.ListUsersDetailed)
    usersAdmin.GET("/:id/full", handlers.GetUserDetailed)
    usersAdmin.POST("", userHandler.Create)
    usersAdmin.PUT("/:id", userHandler.Update)
    usersAdmin.DELETE("/:id", userHandler.Delete)
}

// 📁 CATEGORIES (чтение – гостям, изменения – только админ)
categories := routerv1Public.Group("/categories")
{
    categories.GET("", categoryHandler.GetAll)
    categories.GET("/:id", categoryHandler.GetByID)
}
categoriesAdmin := routerv1Protected.Group("/categories")
categoriesAdmin.Use(adminOnly())
{
    categoriesAdmin.POST("", handlers.CreateCategory)
    categoriesAdmin.PUT("/:id", categoryHandler.Update)
    categoriesAdmin.DELETE("/:id", categoryHandler.Delete)
}

manufacturers := routerv1Public.Group("/manufacturers")
```

```

    {
        manufacturers.GET("", manufacturerHandler.GetAll)
        manufacturers.GET("/:id", manufacturerHandler.GetByID)
    }
    manufacturersAdmin := routerv1Protected.Group("/manufacturers")
    manufacturersAdmin.Use(adminOnly())
    {
        manufacturersAdmin.POST("", handlers.CreateManufacturer)
        manufacturersAdmin.PUT("/:id", manufacturerHandler.Update)
        manufacturersAdmin.DELETE("/:id", manufacturerHandler.Delete)
    }

    // 🍽 БЛЮДА (справочник с составом – для подбора по заказу)
    dishes := routerv1Public.Group("/dishes")
    {
        dishes.GET("", handlers.ListDishes)
        dishes.GET("/:id", handlers.GetDishByID)
    }
    dishesAdmin := routerv1Protected.Group("/dishes")
    dishesAdmin.Use(adminOnly())
    {
        dishesAdmin.POST("", handlers.CreateDish)
        dishesAdmin.PUT("/:id", handlers.UpdateDish)
        dishesAdmin.DELETE("/:id", handlers.DeleteDish)
        dishesAdmin.POST("/:id/products", handlers.AddDishProduct)
        dishesAdmin.DELETE("/:id/products/:dishProductId",
handlers.DeleteDishProduct)
        dishesAdmin.POST("/:id/category-requirements",
handlers.AddDishCategoryRequirement)
        dishesAdmin.PUT("/:id/category-requirements/:reqId",
handlers.UpdateDishCategoryRequirement)
        dishesAdmin.DELETE("/:id/category-requirements/:reqId",
handlers.DeleteDishCategoryRequirement)
    }

    // 📦 PRODUCTS (чтение – гостям, изменения – только админ)
    products := routerv1Public.Group("/products")
    {
        products.GET("", productHandler.GetAll)
        products.GET("/:id/image", handlers.GetProductImage)
        products.GET("/:id", productHandler.GetByID)
    }
    productsAdmin := routerv1Protected.Group("/products")
    productsAdmin.Use(adminOnly())
    {
        productsAdmin.POST("", handlers.CreateProduct)
        productsAdmin.POST("/:id/image", handlers.UploadProductImage)
        productsAdmin.PUT("/:id", handlers.UpdateProduct)
        productsAdmin.DELETE("/:id",
deleteProductAndImage(&productHandler))
    }

    // 📑 ORDERS (требуется авторизации)
    orders := routerv1Protected.Group("/orders")
    {
        orders.GET("", handlers.ListOrders)
        orders.GET("/:id/items", handlers.ListOrderItemsByOrder)
        orders.GET("/:id", handlers.GetOrderByID)
        orders.POST("", handlers.CreateOrder)
        orders.PUT("/:id", handlers.UpdateOrder)
        orders.DELETE("/:id", handlers.DeleteOrder)
    }
}

```

Продолжение Листинга 1

```
// 🛒 ORDER ITEMS (требуется авторизация)
orderItems := routerv1Protected.Group("/order-items")
{
    orderItems.GET("", orderItemHandler.GetAll)
    orderItems.GET("/:id", orderItemHandler.GetByID)
    orderItems.POST("", orderItemHandler.Create)
    orderItems.PUT("/:id", orderItemHandler.Update)
    orderItems.DELETE("/:id", orderItemHandler.Delete)
}

// ❤️ FAVOURITES (только авторизованные пользователи)
favourites := routerv1Protected.Group("/favourites")
{
    favourites.GET("/products", favouriteHandler.ListMyProducts)
    favourites.POST("/product", favouriteHandler.AddProduct)
    favourites.DELETE("/product/:productId",
favouriteHandler.RemoveProduct)
}

// --- GRACEFUL SHUTDOWN ---

srv := &http.Server{
    Addr:    ":8080",
    Handler: router,
}

go func() {
    if err := srv.ListenAndServe(); err != nil && err !=
http.ErrServerClosed {
        utils.Logger.Fatalf("ListenAndServe error: %v", err)
    }
}()

quit := make(chan os.Signal, 1)
signal.Notify(quit, os.Interrupt, syscall.SIGTERM)

<-quit
utils.Logger.Warn("Shutting down server...")

ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
defer cancel()

if err := srv.Shutdown(ctx); err != nil {
    utils.Logger.Fatalf("Server forced to shutdown: %v", err)
}

utils.Logger.Println("Server exiting")
}
```